

RESEARCH ARTICLE

Enhancing DevOps Continuous Monitoring Phase: Hybrid Intrusion Detection and Ensemble Learning System (HIDEELS)

JAFAR ABABNEH¹ , (Member, IEEE), ESAM Y. A. AL-NSOUR¹ ,
ALA'A AL-SHAikh¹, MOHAMMAD RASMI AL-MOUSA¹ ,
AHMAD AL-ZABIN², MAHMOUD ASASSFEH¹,
AND HASAN ABUALESE²

¹Department of Cybersecurity, Faculty of Information Technology, Zarqa University, Zarqa 13110, Jordan

²Prince Abdullah bin Ghazi Faculty of Information and Communication Technology, Al-Balqa Applied University, Al-Salt 19117, Jordan

Corresponding author: Jafar Ababneh (jababneh@zu.edu.jo)

ABSTRACT Companies are excited to maintain high-quality software while reducing the costs of production. DevOps is a contemporary software development life cycle paradigm in which development and operations teams join together during all stages of software development. Nonetheless, security is inadequately integrated inside DevOps. Although there have been attempts to amalgamate security with DevOps, resulting in the emergence of DevSecOps, considerable progress is necessary. The aim of Hybrid Intrusion Detection and Ensemble Learning System (HIDEELS) is to present the incorporation of intrusion detection into the continuous monitoring phase of DevOps, hence enhancing DevSecOps. The integration comprises five machine learning (ML) models and assesses the performance of each model independently. Subsequently, the models are consolidated into an ensemble learning (EL) framework to improve overall robustness and provide more stable predictive outcomes. The results of the individual models show the decision tree (DT) classifier outperforming all the remaining models in terms of accuracy, precision, recall, and f1-score, with 99.5%, 99.5%, 99.7%, and 99.6% on average. Conversely, the EL model attained an average of 99.4%, 99.6%, 99.7%, and 99.7% for accuracy, precision, recall, and F1-score, respectively, exceeding the performance of all other individual ML models.

INDEX TERMS DevOps, DevSecOps, ensemble learning, intrusion detection system, cloud cybersecurity, machine learning.

I. INTRODUCTION

DevOps is a software development method (SDM) in which development (Dev) and operations (Ops) teams collaborate for continuous delivery of software [1]. It aims at the inclusion of all stakeholders in software delivery by integrating the teams, enabling communication between those teams, and eliminating the barriers between them [2]. In general, the DevOps pipeline consists of (1) continuous integration (CI), (2) continuous delivery (CD), (3) continuous deployment, and (4) continuous monitoring (CM) [3].

The associate editor coordinating the review of this manuscript and approving it for publication was Yongming Li .

Continuous monitoring (CM) is the process of discovering vulnerabilities, security issues, compliance requirements, and operational anomalies, and it provides the required feedback about the performance of the system [4]. It is an automated process that can be expanded to discover discrepancies in IT systems. One of the critical roles that CM plays is that it enables rapid evaluation of analytics, as well as quick identification of critical issues.

There is a paucity of research initiatives aimed at enhancing the precision of intrusion detection in DevOps. This work presents a hybrid model Hybrid Intrusion Detection and Ensemble Learning System (HIDEELS) aimed at enhancing the accuracy and performance of intrusion detection

systems (IDS) through machine learning (ML) techniques. An IDS facilitates real-time detection of cyberthreats, thereby enhancing security against both known and unexpected attacks. Accordingly, ML model is proposed which analyzes network traffic to determine whether individual instances of traffic are benign or attacks. Practically, a multitude of machine learning techniques exist for predicting the type of traffic in a network. However, these techniques may vary in different aspects related to the performance measures that are used to evaluate the appropriateness of the desired approach, such as accuracy, precision, recall, and f1-score, to name a few.

HIDEELS employs a machine learning methodology that includes many algorithms: Logistic Regression (LR), k-Nearest Neighbor (kNN), Decision Tree (DT), Naive Bayes Classifier (NB), and Support Vector Machine (SVM). The method evaluates the efficacy of the suggested model during the CM phase. The primary aim is to employ machine learning techniques to analyze security datasets and extract information that can assess the efficacy of machine learning in forecasting the performance of intrusion detection systems.

The integration of machine learning techniques with intrusion detection systems is termed anomaly-based intrusion detection systems. Anomaly-based methods have demonstrated their efficacy in identifying various types of intrusions and attacks. The efficacy of various ML algorithms and their ability to forecast attack types may vary significantly among them. In this scenario, it is challenging to determine which algorithm surpasses the others. An algorithm excels in certain measures while being outperformed by another algorithm in different metrics.

HIDEELS addresses the challenge of selecting the optimal machine learning algorithm by proposing a solution in which four algorithms are executed during the configuration management phase, with the final decision based on the collective results of all four algorithms rather than a single one. The evaluation results are transmitted to the initial stage data to assist in the decision-making process.

We suggest employing an Ensemble Learning (EL) model that consolidates the same machine learning methods utilized individually, and its outcomes is compared with those acquired by each model separately.

A. BACKGROUND

Cybercrime refers to illegal acts or criminal activities that either target or utilize a computer, computer network, or networked device [5]. Cybercriminals, including hackers, individuals, or groups, employ sophisticated methods and possess significant technological expertise to perpetrate cybercrimes. Conversely, some hackers lack experience in the field. Profit, political, or personal motives drive the majority of cybercrimes.

Cybersecurity entails the protection of computer systems and networks from illegal access, theft, or damage. Damage can target either hardware, software, or data. The rising number of Internet users significantly escalates vulnerabilities,

threats, and attacks on information systems, thereby intensifying the demand for innovative strategies to combat these assaults [6], [7].

Confidentiality, integrity, and availability make up the CIA triad, which is a prominent model that serves as the foundation of creating secure systems. Any attack that is targeted against any component of the CIA triad is referred to as an intrusion [8]. When the components of the CIA triad are guaranteed, the security profile of an organization becomes stronger and more prepared to deal with threats. An IDS is hardware or software that monitors a network for hostile activity or policy breaches [9]. In this context, IDS is an application of CM.

Practically, an IDS provides secure, automated threat detection efficiently. It helps decrease the burden of checking networks and servers' functionalities. Additionally, it notifies cybersecurity specialists of any detected suspicious behavior. Artificial intelligence (AI) and machine learning (ML) techniques uncover suspicious activities in anomaly-based IDS. Virtually, those techniques could be looked at as a virtual analyst who utilizes the IDS to protect against different threats and takes the necessary actions when approved. Internally, packet analysis is used to search for attack vectors and classify supervised and unsupervised data [10].

Incorporating ML techniques with intrusion detection entails creating a security model automatically from the training dataset. This dataset includes various data instances that are characterized by features. Those features are also referred to as variables, predictor variables, or independent variables. Furthermore, each data instance is labeled to classify it as benign (or normal) or attack (or anomalous). Data labels are often expressed as binary values, such as 0 for normal and 1 for anomalous. Alternatively, labels can refer to the type of attack, such as DoS, U2R, or R2L. Hence, the ML approach offers details about the different kinds of anomalies.

Experimentally, the existing learning methodologies are insufficiently accurate to identify the specific types of abnormalities. It is highly expensive to obtain a correctly labeled data collection that is representative of all sorts of activities since labeling is frequently done manually by human specialists. Accordingly, three operating modes, namely, supervised learning, unsupervised learning, and semi-supervised learning, are specified for anomaly-based detection algorithms based on the availability of the labels [11].

The application of statistical approaches and ML principles often overlaps. However, the emphasis of ML approaches is on developing models that enhance the performance based on previous data. As a result, IDS exploits ML techniques to improve its execution approach.

HIDEELS employs five machine learning methods to develop a security model for the CM stage of DevOps, specifically:

1) LOGISTIC REGRESSION

LR is an ML classification approach that is utilized to predict the likelihood of a given data instance (or data point)

pertaining to a specific class. When the output of LR is one of two classes, 0 or 1, the model is referred to as a binary LR. Furthermore, there are two kinds of LR, namely, simple LR and multiple (or multivariable) LR. In the former, one predictor predicts the data points' classes; in the latter, there are multiple predictor variables. In essence, the logistic regression model calculates the probability of the outcome using a statistical representation of the relationship between the predictor variable and the predicted variable [12].

2) K-NEAREST NEIGHBOR

kNN is a prevalent machine learning algorithm that employs the proximity between data points, namely distance, in the classification process [13]. The Euclidean distance is a standard metric employed to determine the distances between data points. Notwithstanding the effective performance and simplicity of kNN, a significant limitation is that the choice of the k value profoundly influences the classification quality [14].

3) DECISION TREE

A type of supervised ML is an easy-to-understand method that resembles human reasoning, as it repeatedly divides and tests the data according to a certain parameter [15]. A DT consists of (1) internal nodes, such that each internal node represents a test, (2) a branch that represents the output of a test, and (2) leaf nodes (or terminals) that denote the class labels [16].

4) NAIVE BAYES CLASSIFIER

One of the most straightforward and efficient classification algorithms is NB. The NB classifier performs classification based on the Bayesian theorem [17]. The NB classifier assumes that all features and instances in a dataset are independent of each other [18].

5) SUPPORT VECTOR MACHINE

It is based on the Vapnik–Chervonenkis theory (VC theory), which is a mathematical framework that is used for learning the dependencies from finite samples [19]. Although SVMs are widely used in ML to solve classification problems due to their simplicity and flexibility, there are some caveats related to their computational complexity [20].

B. SECURITY IN DEVOPS

DevOps supports shorter, more controlled iterations by helping businesses strengthen the ties between development, operations teams, and other stakeholders to support the continuous delivery of software versions [21].

Continuous monitoring is a phase of the DevOps process that aims to identify potential infrastructure problems and security threats [22].

CM typically alerts the relevant teams about mistakes made during the development phase, allowing the concerned teams to investigate them.

Not only the development and operations teams must be involved in the DevOps process, but also the security teams must play a pivotal role. The rapid and continuous delivery of software could result in vulnerabilities if security teams are not involved in the DevOps process. Nevertheless, many concerns undermine the adoption of DevOps in the IT sector [24].

Generally, traditional Software Development Life Cycle (SDLC) approaches face numerous issues that render them unsuitable for use in the swift CI/CD of software components. Those traditional approaches consider security a non-functional requirement, which is reviewed at a later stage of the SDLC. While DevOps aims for swift CI/CD of components, postponing security hinders the speed of DevOps [25]. Security in DevOps is referred to as DevSecOps, SecDevOps, SecOps, secure DevOps, safe DevOps, RuggedOps, and security in CD [26], [27], [28]. Through this paper “DevSecOps” term is used to refer to security in DevOps. DevSecOps integrates practices that can prevent security breaches and safeguard information systems. Some of those practices include, but are not limited to, continuous testing, security as code, threat modeling, risk analysis, and red team security drills [27].

Despite the importance of DevSecOps and the important role it provides in producing robust, secure products, DevSecOps still needs more attention to emphasize the role it plays in the different aspects of the product lifecycle [29].

The remainder of this work is structured as follows: In Sect. II, related is discussed to show current research trends, similarities with previous work, as well as research gaps. In Sect. III, we delineate the technique employed in this paper by providing the proposed methodology. Section IV presents and discusses the experimental outcomes. In Sect. V, conclusions and suggest avenues are presented for future research.

II. RELATED WORK

A literature review used IEEE Xplore, Scopus, Web of Science, ACM Digital Library, and Google Scholar. “Intrusion detection system”, “ensemble learning IDS”, “feature selection cybersecurity”, and “machine learning intrusion detection 2019–2025” were used in the search. Inclusion criteria will include peer-reviewed publications between 2019–2025, studies using ML/EL approaches to IDS, and works reporting measurable performance metrics. Exclusion criteria will include non-academic sources, studies lacking evaluation, duplicate surveys, or irrelevant application domains. Of the 203 records identified, 84 were appropriate following title and abstract screening, and 21 met the final evaluation requirements.

For scientific transparency, repeatability, and consistency with scholarly criteria for systematic reviews, Table 1 chronologically summarizes this research by detailing each study's methodology, strengths, and limitations.

Halimaa and Sundarakantham [30] proposed an IDS that is based on SVM and NB. They conducted a comparative

study to evaluate the performance of both models using the NSL-KDD dataset. They performed their experiments on the full dataset without feature selection (FS) and normalization. Their results showed an SVM accuracy of 0.9729 and an NB accuracy of 0.6726. They repeated the results with a subset of the features only selected, and they obtained 0.9395 SVM accuracy and 0.5654 NB accuracy. Finally, they performed normalization and obtained 0.9395 SVM accuracy and 71.001 NB accuracy.

Abrar et al. [31] conducted similar work on some ML models, namely, SVM, kNN, LR, NB, multi-layer Perceptron (MLP), random forest (RF), extra-tree classifier (ETC), and DT. Their work also included a data-preprocessing phase to clean the data and obtain it ready for classification. They also used FS to select different sets of features. The results show that RF, ETC, and DT achieved more than 0.99 performance.

Disha and Waheed [32] also compared some ML models, namely, DT, gradient boosting tree (GBT), MLP, AdaBoost, long-short term memory (LSTM), and gated recurrent unit (GR). They used the UNSW-NB 15 and Network TON_IoT datasets. In their work, they applied FS to select the best-fit features from both datasets. The DT algorithm showed better performance when compared to the other algorithms.

Similarly, Pranto et al. [33] investigated ML-based IDS using the NSL-KDD dataset. They used the following models: kNN, DT, DB, LR, and RF. They also created an EL model from the following four models: kNN, DT, DB, and LR. RF achieved the highest accuracy rate of 0.995. The RF model achieved a higher accuracy rate of 0.995 compared to the EL model's 0.967.

The objectives of Kumar et al. [34] differed from those of the other studies. The three authors conducted their work on the KDD Cup 1999 dataset to study the effects of different normalization schemes on it. Their results revealed that decimal normalization improved the prediction of data points when compared to the same dataset without normalization.

Almaiah et al. [35] studied the use of principal component analysis (PCA) to select features from two different datasets, namely, the KDD Cup 1999 and UNSW-NB15. They used SVM with different kernels. When using the KDD Cup 1999, they obtained 0.99 accuracy, 0.9897 recall, and a 0.9903 F1 score. On the other hand, on the UNSW-NB15 dataset, they recorded 0.94 accuracy, 0.93 recall, and a 0.94 F1 score.

Desai and Gopalakrishnan [36] achieved 0.99 accuracy when using a neural network classifier for an IDS on the Advanced Security Network Metrics (ASNM) dataset.

Elsadig [37] examined the use of ML techniques to detect Denial of Service (DoS) in Wireless Sensor Networks (WSN). Elsadig proposed an approach that utilizes decision trees (DTs) and the Gini feature selection (FS) method. The proposed approach achieved 99.5% accuracy. It was also compared with RF, extreme gradient boosting (XGBoost), and kNN. Not only did the proposed approach outperform the other ML approaches in terms of accuracy, but also in terms

of computation time, which was improved by 9.7% compared to RF, 13% to XGBoost, and 2% compared to kNN.

Fawzy et al. [38] proposes an architecture linking telemetry collection with hybrid detection and automated response. It offers pipeline-aware anomaly detection but remains a prototype without large-scale validation.

Khan et al. [39] aimed at building a network IDS (NIDS) with high accuracy rates using some ML algorithms that showed good results in the literature, namely DT, RF, XGBoost, kNN, NB, LR, and AdaBoost. They conducted a multi-class classification, in which DT outperformed all other algorithms with 96.37% accuracy and a 96.33% F1-score. Then, they experimented with binary classification, and DT scored the best performance in terms of accuracy at 99.96%. The authors also compared the time that is taken to train the models and concluded that binary classification showed better performance in terms of computation time compared to multi-class classification.

Abdulboriy and Shin [40] designed an incremental majority voting approach that encompasses the collaboration of different ML approaches, namely, kNN, Softmax Regressor, Adaptive RF, and Hoeffding Adaptive Tree (HAT). The proposed approach achieved 96.43% accuracy. On an individual basis, kNN achieved the best in terms of all evaluation metrics; it achieved 95.88% accuracy, 95.97% precision, 95.88% recall, and a 95.9% f1-score. The proposed EL model surpassed all the individual models in all evaluation metrics.

Flora et al [41] benchmarks intrusion detection methods in microservice settings, comparing signature, flow, and ML-based approaches. It provides microservice-relevant datasets and metrics, essential for evaluation in DevOps contexts. Limitations include constrained diversity across cloud environments and limited production validation.

Janabi et al., [42] classifies IDS techniques tailored for programmable SDN infrastructures. It emphasizes flow-based and ML approaches useful for DevOps with service meshes. It also highlights unresolved issues such as false positives and limited production deployments.

This paper surveys adversarial ML attacks against IDS, covering poisoning and evasion, and recommends defensive frameworks. It provides threat models but lacks practical deployment case studies in CI/CD settings [43].

Yuan et al. [44] proposes hybrid ensemble ML with adversarial training to harden IDS against evasion. While effective in controlled tests, frequent retraining is resource-intensive for dynamic DevOps contexts.

Olaoye et al. [45] reviews adversarial ML risks for IDS and proposes defense strategies. It offers practical starting points for robustness research but lacks deployment-tested solutions.

Falco (CNCF Project) [46] provides runtime intrusion detection by monitoring syscalls and kernel events with configurable rules enriched by Kubernetes metadata. It is widely adopted in cloud-native DevOps for real-time runtime

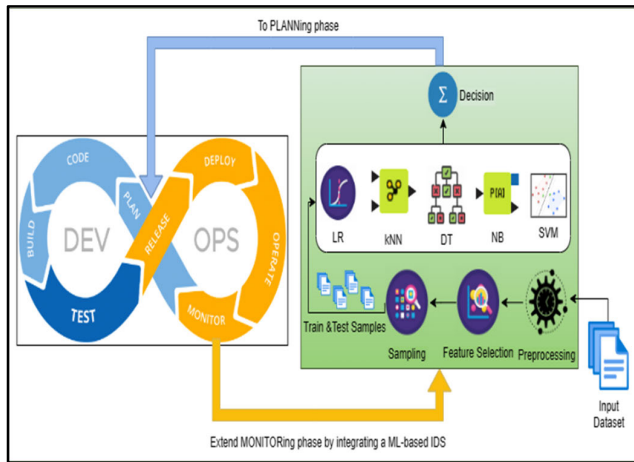


FIGURE 1. The proposed model (HIDEELS).

monitoring. Its benefits include lightweight integration with CI/CD pipelines and operational maturity, though maintaining rules for dynamic microservices is challenging.

Sysdig and Sentinel [47] Industry reports summarizing runtime IDS features, integration with observability platforms, and adoption. They illustrate operational realities but may be biased by vendor perspectives.

In a recent study, Afnan Birahim et al. [48] proposed an EL model that encompasses RF, DT, and kNN. The authors used the Synthetic Minority Oversampling Technique Tomek's (SMOTE-Tomek) techniques to balance the dataset. The proposed EL model achieved an accuracy of 99.73% and a precision, recall, and F1 score of 99.72%. Particle Swarm Optimization (PSO) was used as the FS approach, as well as Grid Search, which was used for hyperparameter optimization. The results of the individual classifiers were 99.68% accuracy for RF, 99.41% for DT, and 98.97% for kNN.

Ness et al. [49] examined the use of a number of ML numbers to detect intrusions in networks; these models are Isolation Forest (IF), NB, XGBoost, Light Gradient-Boosting Machine (LightGBM), SVM, RF, and LR. The results show very poor performance of the IF classifier with an accuracy of 40%. The NB accuracy was 81%, while the accuracy of XGBoost was 83%, and the accuracy of the LightGBM classifier was 100%. The accuracy of the SVM classifier was 85%, 82% accuracy for RF, and 75% accuracy for LR.

Uses ML anomaly detection combining runtime telemetry and CI/CD pipeline events. Improves contextual accuracy but remains an early-stage study with limited real-world deployments [50].

III. PROPOSED METHODOLOGY

Figure 1 delineates the methods employed in this paper, which illustrates the integration of the CM phase of DevSecOps with a machine learning-based Intrusion Detection System (IDS). The IDS presented in this research utilizes

an EL methodology that incorporates five machine learning (ML) techniques: Logistic Regression (LR), k-Nearest Neighbors (kNN), Decision Trees (DT), Naive Bayes (NB), and Support Vector Machines (SVM).

Generally, configuration management commences prior to the software's deployment in production. This paper proposes that CM should inform the relevant teams of errors made during the development phase that jeopardize the product's security. This approach is significant as it facilitates continuous testing of the product for vulnerabilities throughout the SDLC, with further investigations conducted by the relevant teams upon discovery of such vulnerabilities.

In the CM phase, the proposed model consolidates the outcomes of the five machine learning methods and calculates the average of a specified set of performance metrics, including accuracy, precision, recall, and F1-score.

The HIDEELS framework utilizes a soft voting ensemble mechanism, aggregating the probabilistic outputs from five classifiers (logistic regression, k-nearest neighbors, decision tree, naive Bayes, and support vector machine) to select the class with the highest average probability. This method harmonizes the advantages of individual classifiers and mitigates variance relative to rigorous voting.

An occurrence is classified as either an incursion or normal, and the decision is relayed back to the planning phase for management by the appropriate teams.

The main steps for the HIDEELS are as following:

- 1) Cleaning the dataset by removing all the instances that contain null values.
- 2) Removing duplicate instances.
- 3) Normalization.
- 4) Encoding text and categorical features.
- 5) FS.
- 6) Splitting the dataset into a training set and a test set.
- 7) Train each model using the same training set.
- 8) Validate the model results using the selected evaluation metrics.
- 9) At this point, the models ready to start making predictions of new instances.

The flow chart of the proposed model is given in Fig. 2.

A. PREPROCESSING STEPS

Any study involving data analysis must begin with the preprocessing phase. The elimination of null values and duplicate instances is one of several procedures in preprocessing that helps keep ML models efficient and prevents them from making inaccurate predictions.

B. NORMALIZATION

Normalization is utilized to ensure that the value ranges of each feature are closely aligned, predominantly within the interval [0, 1]. It assists in averting bias in the model towards elevated values, so guaranteeing that all values possess an equal chance of being incorporated into the model, which subsequently influences the model's overall effectiveness.

TABLE 1. A comparison between the recent state of the art studies.

Ref.	Year	Method(s)	Benefits	Limitation
Halimaa and Sundarakantham [30]	2019	SVM and NB	Used a well-organized classification Methodology to reduce false alarms and to increase the detection rate.	- Used only two ML algorithms, namely, SVM and NB, where there are other algorithms that were never investigated and can achieve better results. - The performance of an EL model was not measured.
Abrar et al. [31]	2020	SVM, kNN, LR, NB, MLP, RF, ETC, and DT	- Used more ML algorithms than the previous work (8 algorithms). - Used four feature subsets from the dataset. - The computational complexity was reduced. - The prediction rate was improved.	- The performance was not measured as an EL model. - It is not clear which FS approach was used to select the four subsets of the dataset.
Disha and Waheed [32]	2022	DT, GBT, MLP, AdaBoost, LSTM, and GR	- Used different ML algorithms. - Used embedded FS that was based on the importance score. - Compared the performance with the related work using the f1-score which shows resilience.	- Did not collate the algorithms in an EL model. - Did not present the amount of time improvement due to FS.
Pranto et al. [33]	2022	kNN, DT, DB, LR, and RF	- Used an EL model based on 4 ML algorithms that were used in the study. - Used FS.	- The accuracy of one individual model was better than that of the EL model. - Used a rule-based FS that is based on only two rules, one for correlation and the other is for variance. Thus, adding more rules could improve the results. Also using a wrapper FS approach could give better results. - No EL model was created. - FS was not employed.
Kumar, Gupta, and Arora [34]	2022	SVM, DT, RF, kNN, NB, and MLP	Used different normalization schemes on the dataset to measure how they affect the performance of intrusion detection.	- Used only one ML algorithm. Thus, there was no comparison between different ML approaches. - Used PCA only for FS, whereas different FS approaches can be used and compared. - Did not compare PCA to wrapper methods.
Almaiah et al. [35]	2022	SVM	Studied the effect of FS on the classification results.	- Used only variance threshold and Chi-square Test for FS. - No EL model was employed.
Desai and Gopalakrishnan [36]	2023	neural network classifier	- Using a neural network in the intrusion detection. - FS was applied. 99% accuracy rate was achieved.	- The work is only focused in WSN while it can also study IoT and general purpose computer networks. - The work focuses only on DoS attacks. - Prototype focus; lacks scale tests
Elsadig [37]	2023	DT, RF, XGBoost, and kKNN	- Studied DoS attacks. - The work is focused around WSNs. - Used FS. - Used EL. - Pipeline-aware, automated alerting/mitigation	EL was not employed.
[39] Fawzy et al., DADF Framework	2023	Hybrid anomaly detection architecture with telemetry ingestion	- The dataset was balanced using SMOTE. - FS was employed. - EL was employed. - Reduced false alarm rates. - FS was employed. - Microservice-relevant metrics; useful for ephemeral/east-west traffic	- Could have used better dataset balancing techniques such as SMOTE. - Limited cloud diversity; not large-scale
Khan et al [40]	2024	DT, RF, XGBoost, kNN, NB, LR, and AdaBoost		
Abdulboriy and Shin [39]	2024	EL using kNN, Softmax Regressor, Adaptive RF, and HAT		
[41] J. Flora et al., Microservice IDS Benchmark	2024	Benchmark datasets and experiments (signature, flow, ML)		
[42] Janabi et al., SDN IDS Survey	2024	Survey of flow-based, controller-hardening, ML IDS in SDN	Clarifies SDN-specific telemetry & taxonomy	Few production validations; high false positives
[43] Adversarial Challenges in NIDS (Survey)	2024	Survey of adversarial ML attacks (poisoning, evasion)	Threat modeling, defense strategies	Lacks operational validation

TABLE 1. (continued.) A comparison between the recent state of the art studies.

[44] X. Yuan et al., Robust IDS	2024	Hybrid ensemble ML with adversarial training	• Improved robustness against evasion attacks	• Training cost; reduced benign accuracy
[45] Olaoye et al., Adversarial ML for IDS	2024	Review + experiments of adversarial attacks and defenses	• Defensive strategies (robust features, ensembles)	• Prototype/preprint; lacks deployment detail
[46] Falco (CNCf Project)	2024 – 2025	Rule-based syscall/kernel-event monitoring, Kubernetes metadata enrichment	• Real-time runtime detection; integrates with CI/CD; lightweight agent	• Rule maintenance; may miss novel attacks
[47] Vendor Reports (Sysdig, SentinelOne)	2024 – 2025	Market/feature analysis of runtime IDS tools	• Shows adoption, integration with observability tools	• Vendor bias; hidden costs/performance
Birahim et al. [48]	2025	EL using RF, DT, and kNN	• Used EL. • Used PSO for hyperparameter tuning. • Used SMOTE for balancing the dataset.	• Used manual FS. • Increased computational complexity of the proposed model.
Ness et al. [49]	2025	IF, NB, XGBoost, LightGBM, SVM, RF, and LR	• Explored supervised and unsupervised approaches. • Used SMOTE for balancing the dataset.	• Used PCA for FS which could be improved if wrapper-based FS was employed. • Did not employ an EL approach based on the models that were tested.
[50] ML Anomaly Detection in Containers (Study) Proposed Model (HIDEELS)	2025	ML on container telemetry + CI/CD event correlation	• Context-aware detection of pipeline anomalies	• Small-scale study; labeling challenges
			This work introduces intrusion detection to the continuous monitoring phase of DevOps, boosting DevSecOps. Five machine learning (ML) models are integrated and evaluated separately. To improve results, the models are combined into an ensemble learning (EL) framework and it shown promising empirical outcomes.	Not all ML models are implemented; artificial neural networks (ANN). Also, more the machine learning models parameters where not set using hyper parameters optimizations which will enhance the results

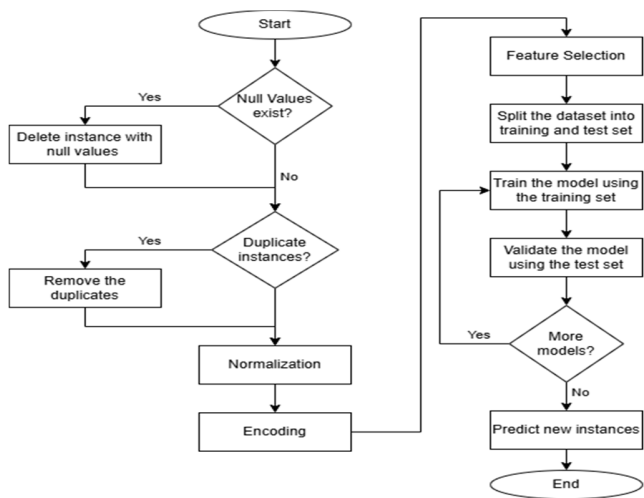


FIGURE 2. Flowchart of the proposed model (HIDEELS).

C. ENCODING OF TEXT AND CATEGORICAL FEATURES

Many textual and category attributes in the chosen dataset make conventional machine learning models, designed for

numerical data processing, incompatible. The decision to remove those features may adversely affect the model’s overall performance and could result in degradation, particularly if the information contained inside them is significant. The decision is to convert those traits into numerical values for subsequent processing by the models. The decision to retain or delete them will be made during the subsequent FS stage.

Two encoding methodologies are employed: one-hot encoding (OHE) and label encoding (LBE). The One-Hot Encoding (OHE) approach is more appropriate for encoding nominal data that lacks any relational hierarchy, such as ranking or order. The outcome of One-Hot Encoding (OHE) is a set of binary features corresponding to the number of unique values in the feature requiring encoding. The values of each emergent feature are assigned as 1 if the instance possesses that value for the original feature; otherwise, they are designated as 0. Although one-hot encoding (OHE) may enhance the model, it can also elevate the dataset’s dimensionality, thus augment the model’s complexity and impede the pace of training and testing.

The second encoding system employed in this research is LBE, which is inherently suitable for ordinal data or data with

a defined order. In contrast to OHE, LBE does not alter the dimensionality of the dataset, hence ensuring that the model's complexity remains unaffected by LBE, provided that LBE does not invoke the curse of dimensionality. Conversely, the outcome of LBE is a singular novel feature with integer values corresponding to each ordinal.

Upon completion of the encoding step, regardless of whether One-Hot Encoding (OHE) or Label Encoding (LBE) is employed, the original feature is eliminated from the feature space, but the newly generated features are retained. Nonetheless, the dataset's dimensionality may expand, making feature selection essential for diminishing dimensionality by eliminating unnecessary information, hence reducing the complexity of the machine learning model. Consequently, the model's performance is enhanced regarding both runtime and accuracy.

D. FEATURE SELECTION

There are three primary categories of FS, specifically. The first category is filter-based feature selection, which uses statistical methods to separately assess the correlation between each feature and the target variable. In this approach of feature selection, neither model training nor testing occurs throughout the feature selection process. Consequently, the filter-based methodology is both straightforward and efficient. The second method is wrapper-based feature selection, which conducts model training and testing by including or excluding specific characteristics or subsets of features. This type of feature selection employs a machine learning model that operates on a subset of features to assess performance. Ultimately, the features that produce optimal performance are identified as the superior feature set. The final category is the embedded FS methodology. In this third strategy, each feature is assigned a relevance score during the model's training. This score is utilized to discard elements of lesser significance and retain those of greater relevance. Thus, embedded approaches exhibit superior speed compared to the wrapper approach.

This research employs two feature selection methodologies: SelectFromModel and recursive feature elimination (RFE). SelectFromModel is an embedded feature selection method that assigns weights to features, indicating their significance. It employs an estimator, a machine learning method, that conducts estimations on features during the training process. A threshold is established, for instance, the average of feature significance. Only features with importance equal to or exceeding the threshold are selected, and those below it is discarded. The second feature selection method employed in this paper is Recursive Feature Elimination (RFE), a wrapper-based technique. It operates by repeatedly evaluating the significance of each element. The method operates by establishing a machine learning model that evaluates the significance of the features. Features with diminished significance values are systematically removed from the dataset. HIDEELS model is configured to execute both FS algorithms and evaluated their performance, selecting the one that exhibits superior results.

E. EVALUATION OF THE MODEL'S PERFORMANCE

After training and testing the models, the individual models' performance is computed. The models are then aggregated in an EL model. The performance of the EL is also computed. The performance of each model is computed in terms of accuracy, precision, recall, and F1 score.

Accuracy is the ratio of correctly predicted values to the total values and is computed according to (1) [51].

$$acc = \frac{TP + TN}{TP + FP + TN + FN} \quad (1)$$

such that, TP is the number of positive values that were correctly predicted as positive, TN is the number of negative values that were correctly predicted as negative, FP is the negative values that were incorrectly predicted as positive, and FN is the positive values that were incorrectly predicted as negative.

Precision (p) is another metric that is sometimes referred to as positive predictive value and it shows the ability of a classifier to predict positive values as positive. It is computed according to Eq. 2 [52].

$$p = \frac{TP}{TP + FP} \quad (2)$$

As shown in (2), p is reversely proportional to FP , which implies that higher p values indicate that the model is less vulnerable to classifying negative values as positive.

Recall (r) is also referred to as true positive rate (TPR) or sensitivity. It measures the rate of the positive values that were predicted correctly. The recall metric shows the sensitivity, or tendency, of the model to predict positive values. Recall is computed according to (3).

$$r = \frac{TP}{TP + FN} \quad (3)$$

Precision and recall indicate the capability of the model to detect positive values [52]. Nevertheless, the trade-off between precision and recall is a debate. Thus, F1-score (f_1) is computed to resolve the conflict between the two measures and is computed according to (4) [52].

$$f_1 = 2 \times \frac{p \times r}{p + r} \quad (4)$$

F. CROSS-VALIDATION

To check the robustness and the reliability of the proposed approach, K-fold cross-validation is used at the end of the experiments. The main idea of K-fold cross-validation is to split the dataset into subsets (or folds); the model is trained on folds, those are referred to as the training sets, and the remaining fold is used for testing and validation and is referred to as the validation set. The split is given in (5):

$$D = \bigcup_{i=1}^K D_i \quad (5)$$

The process is repeated K times, such that at each iteration k , the subset D_k is used as the validation set, and the remaining

sets are the used as the training sets. The collection of the training sets in the k^{th} iteration $D_{train}^{(k)}$ is given in (6):

$$D_{train}^{(k)} = \left(\bigcup_{i=1}^K D_i \right) - D_k \quad (6)$$

At the end of the process, the evaluation metric M is averaged to find the overall performance of the system, as shown in (7).

$$CV(K) = \frac{1}{K} \sum_{i=1}^K score(M_i, D_i) \quad (7)$$

Such that $CV(K)$ is the cross-validation performance (or score) at the end of K iterations, M_i is the i^{th} model which is trained on the $D_{train}^{(i)}$ dataset against the D_i validation set.

IV. EXPERIMENTAL RESULTS

A. EXPERIMENTAL ENVIRONMENT

The experiments were set to run on Google Colab. The machine that was allocated by Google Colab for those experiments was coded in Python 3 Google Compute Engine. The engine is equipped with an x86-64 processor, 12.7 GB RAM, and a 107.7 GB disk and runs the Linux operating system. The ML models were built using Python 3.8.10.

The machine that was used to access Colab is equipped with an 11th generation Intel®Core (TM) i7-1165G7 at 2.80 GHz speed. The processor is a 4-core CPU with Hyper-Threading technology, which amounts to 8 cores. The processor has a 12-MB Intel®Smart Cache. The amount of RAM installed is 8 GB. The system runs a Windows 11 Pro operating system.

B. EXPERIMENTAL SETUP

The KDD Cup 1999 dataset is used [53] to test and evaluate HIDEELS system. Research extensively uses the KDD Cup 1999 dataset to test and evaluate the performance of ML-based IDS. The dataset consists of 42 features and contains nearly half a million instances of network traffic. The instances are labeled, or classified, as 22 different types of attacks, or intrusions, as well as one non-intrusion, or benign, traffic that is referred to as normal, which totals 23 types of traffic. The cleaning step reduces the number of instances in the dataset to nearly 150K. The preprocessing phase reduced the number of instances in the dataset by more than 70%, which, in turn, is expected to improve the runtime and prediction efficacy of the ML models. Fig. 3's tree map illustrates the distribution of the 23 distinct types of traffic in the KDD Cup 1999 dataset. Fig. 3 clearly demonstrates the imbalance in the dataset.

Normal traffic forms more than 50% of the traffic. On the other hand, Neptune attacks form more than one-third of the traffic, and the other kinds of attacks share 5% of the traffic.

Despite the imbalanced nature of the dataset, it accurately represents the type of traffic encountered on the Internet, where normal traffic makes up the majority. Typically, most Internet users are not bad actors; still, there is a small portion of users who are bad actors.

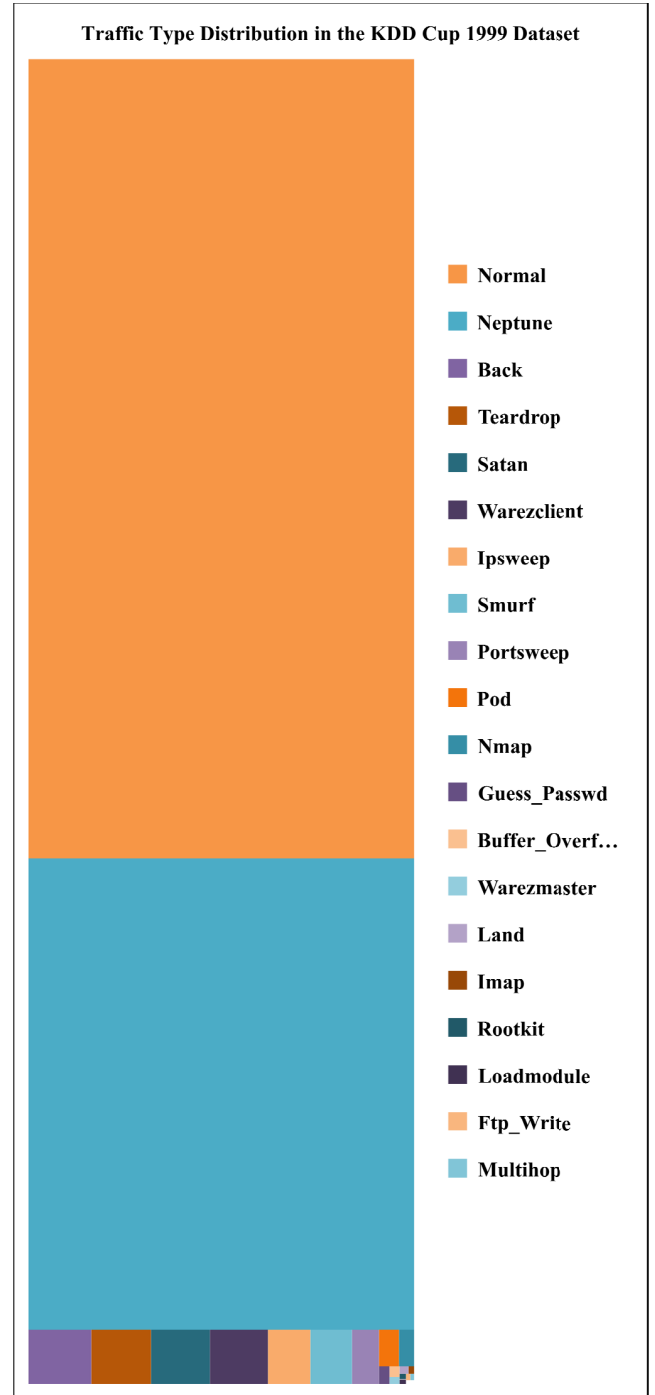


FIGURE 3. Traffic type distribution in the KDD Cup 1999 dataset.

In practice, imbalanced datasets can lead to the system failing to accurately predict the class of data points that belong to the minority class. In the case of the KDD Cup 1999 dataset, normal traffic instances dominate the dataset, which increases the chance of incorrectly detecting new instances as normal. Accordingly, accuracy would be insufficient to describe the performance of the ML approach; precision, recall, and f1-score must be used in conjunction with accuracy to judge the efficacy of the model.

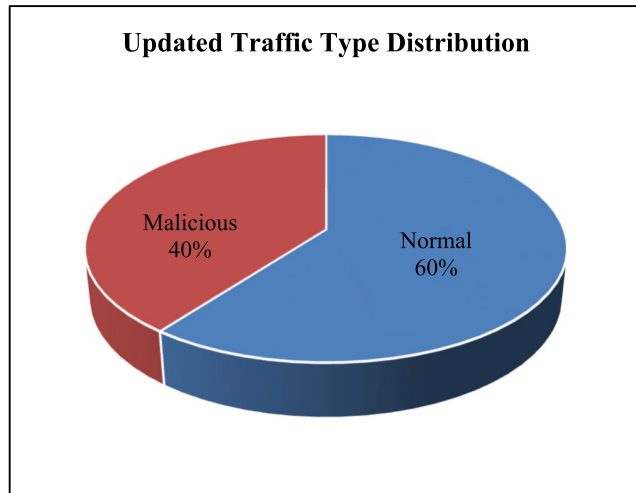


FIGURE 4. New traffic type distribution after updating the dataset to include normal and malicious traffic.

To improve the predictions, class consolidation (or merging) is performed and all non-normal entries in the target variable of the dataset is updated to one new type that we called malicious. Accordingly, then divided the traffic into two classes only, namely, normal and malicious. After carrying out this update, the distribution of the target variable would be as shown in Fig. 4. Nearly 60% of the traffic is normal, and the remainder is malicious, which gives more balance to the dataset.

The subsequent stage in the preprocessing phase is normalization, which seeks to reduce the range of each feature, specifically the disparity between the highest and minimum values of a feature. Table 2 enumerates the attributes of the KDD Cup 1999 dataset that underwent normalization. The ranges of certain features, such as duration, src_bytes, and dst_bytes, are excessively high.

Furthermore, some characteristics with ranges are excessively low, including wrong_fragment, num_failed_logins, and su_attempted, among others.

The features that have excessively high ranges will influence the outcomes of the ML model, making them appear to be the most significant characteristics, even if they are not.

Normalization is conducted to standardize the ranges of all features to a uniform scale, specifically 1, as illustrated in Table 2. As a result, no feature predominates over the others during the model training process. The outcome is enhanced performance regarding both accuracy and training duration.

The last step in the preprocessing phase is the encoding. The encoding step aims to convert the non-numeric, or categorical, features into numeric ones. Table 3 lists the categorical features of the KDD Cup 1999 dataset. One-hot encoding (OHE) to encode the independent variables is used, namely, protocol_type, service, and flag. One-hot encoding creates an independent variable for each unique value from the set of values of the original non-numeric feature.

The emergent variables are named according to their values, and only the variable whose name matches the value that

TABLE 2. The numeric features with high ranges before and after normalization.

Feature	Before Normalization		After Normalization	
	Datatype	Range	Datatype	Range
Duration	Integer	58329	Float	1
src_bytes	Integer	693375640	Float	1
dst_bytes	Integer	5155468	Float	1
wrong_fragment	Integer	3	Float	1
Urgent	Integer	3	Float	1
Hot	Integer	30	Float	1
num_failed_logins	Integer	5	Float	1
num_compromised	Integer	884	Float	1
su_attempted	Integer	2	Float	1
num_root	Integer	993	Float	1
num_file_creations	Integer	28	Float	1
num_shells	Integer	2	Float	1
num_access_files	Integer	8	Float	1
Count	Integer	511	Float	1
srv_count	Integer	511	Float	1
dst_host_count	Integer	255	Float	1
dst_host_srv_count	Integer	255	Float	1

is stored in the original feature is set to 1, while the other variables are set to 0. After performing OHE on the first three features listed in Table 3, then the original three from the dataset is removed. As a result, the KDD Cup 1999 will contain 119 features after finishing the OHE step.

The last categorical feature is the dependent, or target, variable itself, which is traffic_type. Label encoding (LBE) is used to encode the target variable traffic_type. Our choice resulted in encoding normal traffic with 1 and malicious traffic with 0.

It is crucial to observe that, in contrast to One-Hot Encoding (OHE), Label Encoding (LBE) does not augment the dataset with additional features; it only encodes each feature value with an integer. In this context, it is essential to elucidate the preference for employing OHE rather than LBE for the independent variables. Encoding a feature with an integer implies an ordinal relationship or quantity, potentially influencing classifier performance and resulting in erroneous outcomes. The values of the dependent variable, or target, signify the classes of the data points, elucidating our preference for LBE over OHE in encoding the target variable.

The extensive array of characteristics produced during the preprocessing phase presents several challenges. It is indisputable that the 119 features will be comprehensive for the model's runtime. Consequently, an increase in the number of features inside the model correlates with an extended duration required for training, testing, and prediction. The second issue is overfitting, which elevates the model's variance and thus diminishes its efficacy [54]. Ultimately, an excessive number of features will augment the noise within the dataset,

TABLE 3. List of categorical variables and their encoding methods.

Feature	Datatype	Unique Values	Values	Encoding	Result
protocol_type	Text	3	icmp, tcp, udp		+3 features
Service	Text	66	IRC, X11, Z39_50, auth, ...	OHE	+66 features
Flag	Text	11	OTH, REJ, RSTO, RSTOS0, ...		+11 features
traffic_type	Text	2	Normal, Malicious	LBE	1 - Normal, Malicious

comprising elements that are inconsequential to the model, hence diminishing its accuracy.

Therefore, the significance of the FS phase emerges as a crucial stage in the preparation of the dataset for utilization by the ML model. Eliminating extraneous features from the dataset addresses the aforementioned issues, hence enhancing the model's runtime, mitigating overfitting, and increasing the accuracy of the machine learning model [55].

C. DISCUSSION AND ANALYSIS

We compared the performance of two well-known FS schemes, namely, SelectFromModel and recursive feature elimination (RFE) ar. SelectFromModel is a meta-transformer that works in conjunction with a classifier, or an estimator [56], to select the most important features from the dataset in a way that achieves the best accuracy. The RFE is an iterative FS approach that, like SelectFromModel, works in conjunction with a classifier and iteratively selects a set of features, trains the model, fits it, predicts, and measures the performance, then removes the features with the least ranks [57].

The results are summarized in Table 4, where SelectFromModel is much faster in selecting the same number of features as RFE then computing the enhancement achieved by the runtime of the SelectFromModel FS approach over the RFE approach. Let be the runtime of the faster approach, and be the runtime of the slower approach; then the enhancement achieved by the faster approach compared to the slower approach is computed using (8).

$$E = \left(1 - \frac{T_f}{T_s}\right) \times 100\% \quad (8)$$

According to Eq. (8), FS using SelectFromModel achieves more than 93% enhancement over the RFE approach. Consequently, a decision was made to use the SelectFromModel FS approach to select the most important 34 features from the KDD Cup 1999 dataset.

Prior to commencing model construction, the penultimate stage in the dataset preparation phase is the sampling phase. We generate predefined samples, or subsets, of the dataset with the following dimensions: 1k, 5k, 10k, 50k, 100, and the complete dataset, comprising 145,586 data points, henceforth

TABLE 4. Runtime comparison between selectfrommodel and RFE.

Model	Selected Features	Elapsed Time
SelectFromModel	34	32.55
RFE	34	467.65
Enhancement		93.04%

TABLE 5. Sample sizes and the proportion of each class of traffic type within each sample.

Traffic Type	No. of Data points	% Data points	Sample Size 1k	5k	10k	50k	100k	Full
Normal	87832	60.33	603	3016	6033	30165	60330	87832
Malicious	57754	39.67	397	1984	3967	19835	39670	57754

referred to as “full.” To maintain consistent representation of each traffic type class in every sample, samples from each class according to their proportion in the complete dataset are selected, as illustrated in Table 5.

The objective of generating preset samples is to mitigate the influence of unpredictability on the models.

When all models are executed on identical dataset samples, we ensure that no model, by chance, receives a greater number of discriminative data points than the others, thereby preventing superior outcomes attributable to this randomness factor.

Initially, each model separately is assessed. Every model undergoes training and testing on each sample. For each model, the runtime (t), accuracy (acc), precision (p), recall (r), and F1 score (f_1) are calculated. The values are enumerated in Table 6.

Table 6 indicates that Naive Bayes (NB) surpasses Decision Trees (DT), Logistic Regression (LR), k-Nearest Neighbors (kNN), and Support Vector Machines (SVM) regarding runtime efficiency. Conversely, SVM exhibits the longest runtime, succeeded by kNN.

Figure 5 illustrates a runtime comparison between the two models with optimal performance, specifically NB and DT, highlighting the superior speed of NB relative to DT, which ranks second in runtime efficiency.

The most accurate results were obtained using DT with a sample size of 10,000. However, when tested on the entire dataset, DT also produced the greatest results in terms of recall, F1 score, and precision. The accuracy rates of all the models are compared in Figure 5. In terms of accuracy, DT surpasses all of the models, as demonstrated in Figure 5. In terms of accuracy, kNN is the second-best model, followed by SVM and LR, and NB is at the bottom.

As illustrated in Figure 6, the models have commendable accuracy performance. By preprocessing the dataset to classify data points into only two categories—normal and malicious—we clearly achieved better performance results and added more balance. This was true even though the dataset was imbalanced to begin with.

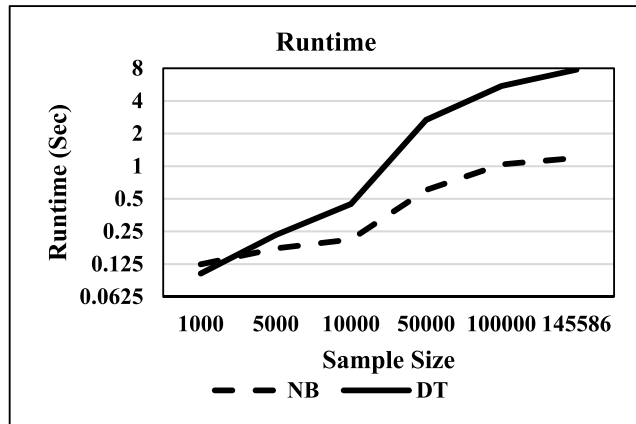


FIGURE 5. Runtime comparison between NB and DT.

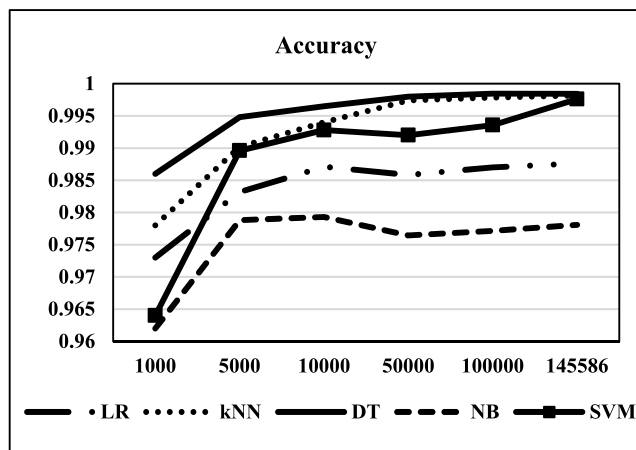


FIGURE 6. Comparison of the accuracy rates of the five models.

Additionally, evaluated the models' performance regarding precision are evaluated, as illustrated in Fig. 7. The results indicate that DT surpasses all other models in precision across all sample sizes, followed by kNN, SVM, LR, and lastly, NB.

Given that precision is inversely related to false positives (FP), defined as negative instances erroneously classified as positive, the results depicted in Fig. 6 indicate that Decision Trees (DT), k-Nearest Neighbors (kNN), and Support Vector Machines (SVM) generated the fewest FP predictions. LR and NB exhibited a higher frequency of false positive predictions compared to the other three models, indicating that both were less precise in traffic classification.

Figure 8 illustrates a comparison of the five models regarding recall. Figure 8 clearly illustrates that the models exhibit varying behaviors about recollection. In contrast to accuracy and precision, which yielded highly discriminative findings, the recall results did not. The recall values for DT, kNN, and SVM exhibit proximity, occasionally demonstrating alternate patterns where DT initially surpasses the others, followed by kNN, then SVM, with DT ultimately excelling in bigger sample sets. The values attained by LR are not significantly different from those obtained by the preceding three models.

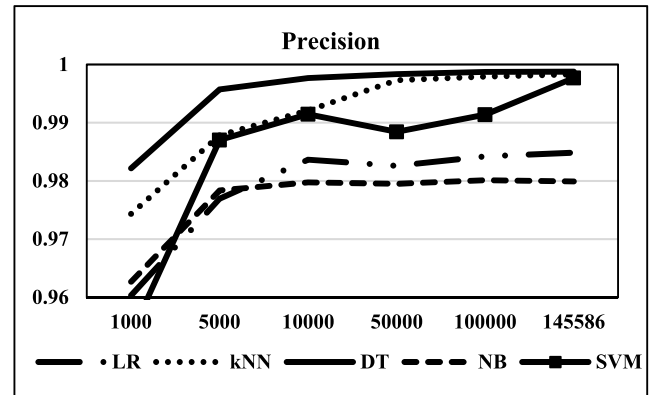


FIGURE 7. Comparison of the accuracy rates of the five models.

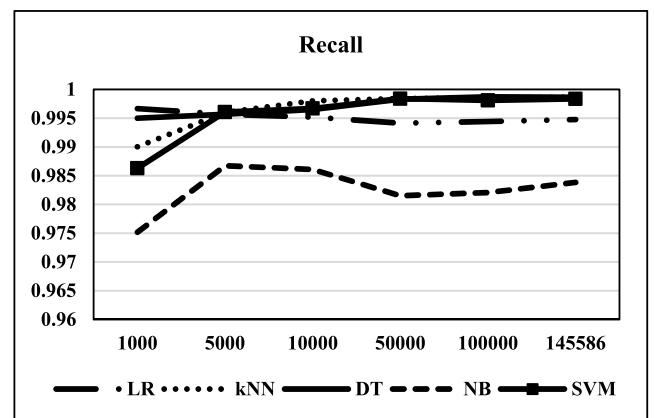


FIGURE 8. Comparison of the recall rates among the five models.

Since recall assesses the model's propensity to anticipate positive values, the findings depicted in Fig. 8 indicate that all models exhibit a strong inclination to predict positive outcomes. Once again, NB exhibits a diminished propensity to forecast positive values in comparison to the other models. Figure 9 illustrates the comparative F1 scores of the five models. The balance between precision and recall has consistently posed a challenge; hence, the F1 score serves as a resolution to this dilemma. Figure 8 illustrates that DT surpasses the other models in f1-score, succeeded by kNN, SVM, LR, and ultimately NB.

The average runtime, accuracy, precision, recall, and F1-score for each model separately are calculated. The peak value of each parameter is subsequently documented in the bottom row of Table 7. This maximum value will then be utilized for comparison with the EL model. The findings indicate that NB surpasses the other models for average runtime. In terms of mean accuracy, precision, recall, and F1-score, decision trees outperform the others.

Furthermore, Figure 10 presents a graphical comparison of the five models which indicates that no model surpasses DT across all average metrics. Subsequently, kNN is succeeded by SVM, followed by LR. The model that is consistently guided by the other models is NB.

TABLE 6. Computation time, accuracy, precision, recall, and F1-score for each ML model on each dataset sample.

Model	Sample Size	<i>t</i>	<i>acc</i>	<i>p</i>	<i>r</i>	<i>f</i> ₁
LR	1K	0.249999	97.3000%	96.0310%	99.6667%	97.8087%
	5K	0.49694	98.3200%	97.6951%	99.5684%	98.6210%
	10K	0.949074	98.7100%	98.3634%	99.5193%	98.9375%
	50K	5.97688	98.5820%	98.2578%	99.4132%	98.8319%
	100K	17.9756	98.7010%	98.4221%	99.4414%	98.9290%
	Full	23.8702	98.7609%	98.4852%	99.4763%	98.9782%
kNN	1K	0.201222	97.8000%	97.4335%	99.0027%	98.1976%
	5K	0.865091	99.0200%	98.7843%	99.6015%	99.1906%
	10K	3.35306	99.4000%	99.2100%	99.8011%	99.5044%
	50K	50.7802	99.7400%	99.7286%	99.8409%	99.7847%
	100K	195.964	99.7840%	99.7913%	99.8508%	99.8210%
	Full	417.77	99.8159%	99.8304%	99.8645%	99.8475%
DT	1K	0.102282	98.6000%	98.2169%	99.5000%	98.8481%
	5K	0.232194	99.4800%	99.5722%	99.5689%	99.5696%
	10K	0.448056	99.6500%	99.7679%	99.6519%	99.7096%
	50K	2.67039	99.7980%	99.8343%	99.8309%	99.8326%
	100K	5.4858	99.8450%	99.8724%	99.8707%	99.8715%
	Full	7.7616	99.8434%	99.8770%	99.8634%	99.8702%
NB	1K	0.124394	96.2000%	96.2675%	97.5164%	96.8754%
	5K	0.174076	97.8800%	97.8358%	98.6732%	98.2502%
	10K	0.211168	97.9300%	97.9758%	98.6077%	98.2901%
	50K	0.603815	97.6440%	97.9502%	98.1502%	98.0494%
	100K	1.04055	97.7160%	98.0123%	98.2065%	98.1090%
	Full	1.19034	97.8068%	97.9896%	98.3833%	98.1860%
SVM	1K	0.139033	96.4000%	95.3642%	98.6301%	96.9697%
	5K	0.800045	98.9600%	98.7030%	99.6073%	99.1531%
	10K	2.19147	99.2800%	99.1453%	99.6695%	99.4067%
	50K	43.0541	99.2000%	98.8430%	99.8406%	99.3393%
	100K	897.679	99.3560%	99.1385%	99.8094%	99.4728%
	Full	862.462	99.7610%	99.7687%	99.8366%	99.8026%

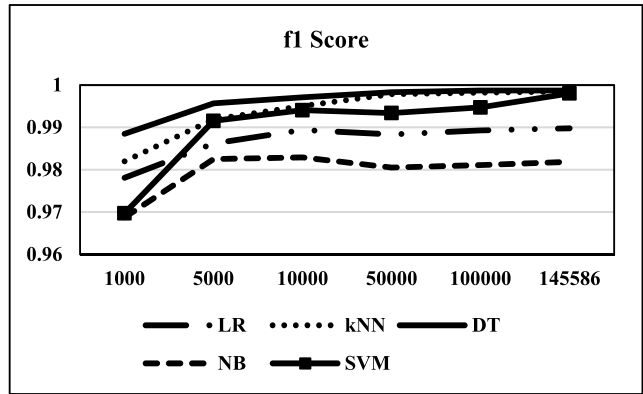


FIGURE 9. Comparison of the f1-score rates of the five models.

The results indicate that certain models exhibit superior performance compared to others. To enhance the predictive accuracy of all models, an ensemble learning approach is developed which integrates the five models into a single ensemble model. K-fold cross-validation is employed to develop the ensemble model comprising five estimators: LR, kNN, DT, NB, and SVM. The EL system model is designed to operate on identical samples to ensure that the findings are objective, facilitating unbiased comparisons between the individual ML models and the aggregated EL model.

The number $k=10$ is established to conduct a robust cross-validation, as lower values may result in an underestimation

TABLE 7. Computation time, accuracy, precision, recall, and F1-score for all models.

Model	<i>t</i>	<i>acc</i>	<i>p</i>	<i>r</i>	<i>f</i> ₁
LR	8.253116	98.3957%	97.8758%	99.5142%	98.6844%
kNN	111.4889	99.2600%	99.1297%	99.6603%	99.3910%
DT	2.783387	99.5361%	99.5235%	99.7143%	99.6169%
NB	0.557391	97.5295%	97.6719%	98.2562%	97.9600%
SVM	301.0543	98.8262%	98.4938%	99.5656%	99.0240%
Max	301.0543	99.5361%	99.5235%	99.7143%	99.6169%

of the model. Cross-validation is conducted for each EL model across various sample sizes, with the results documented in Table 8. The mean value of the cross-validation is calculated and documented in the bottom row of Table 8 as well. The EL system demonstrates an average runtime of around 328 seconds, with an average accuracy of 99.4%, an average precision of 99.6%, an average recall of 99.7%, and an average F1-score of 99.7%.

The findings from Table 8 in Fig. 10 is illustrated to compare the performance of the models. Figure 10 illustrates that the EL model surpasses the individual models regarding average accuracy, precision, recall, and F1-score.

Figure 10 demonstrates that developing an ensemble learning model that integrates many machine learning models yields superior outcomes compared to the performance of each model in isolation.

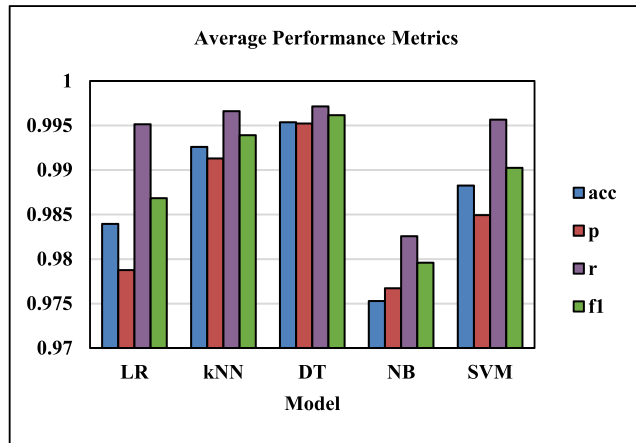


FIGURE 10. Comparative analysis of the five models regarding average accuracy, precision, recall, and F1-score.

TABLE 8. Performance of the EL model on the identical samples utilized with the individual ML model.

Sample size	<i>t</i>	<i>acc</i>	<i>p</i>	<i>r</i>	<i>f₁</i>
1K	0.519739	98.4000%	99.3329%	99.3972%	99.3650%
5K	1.88149	99.0060%	99.5870%	99.4735%	99.5302%
10K	5.79571	99.5820%	99.5895%	99.7876%	99.6884%
50K	52.4222	99.6300%	99.5952%	99.8992%	99.7470%
100K	961.871	99.7960%	99.6863%	99.9226%	99.8043%
Full	942.514	99.8903%	99.7561%	99.9292%	99.8426%
Average	327.5007	99.3840%	99.5912%	99.7349%	99.6629%

Table 9 presents the mean of all performance measures for the five distinct models, together termed “All,” alongside the EL model, designated as “EL.”

The experimental results indicated that certain machine learning models surpassed others, with some models significantly outperforming the rest. Nonetheless, when all models were consolidated under an ensemble learning framework, the subpar performance of certain models was alleviated and enhanced by the superior performance of others.

The performance of the model was also tested using the raw network packets of the UNSW-NB 15 dataset [58], [59], [60], [61], and [62], whose records were collected using the tcpdump tool which was set to capture 100 GB of traffic. The dataset contains 49 features including the class label, and contains nearly 2.5 million records that are distributed as shown in Figure 11.

The majority of the dataset, that is 87% of it, is normal traffic, which indicates that the dataset is unbalanced. The dataset is split into two subsets by the creators of the dataset. The first subset contains 175,341 records is the training set, and the other subset is of 82,332 records and is used as a testing set. Both the training and testing sets contain normal and malicious traffic that are distributed as shown in Figure 12. The dataset was undergone some preprocessing steps in order to prepare it for analysis and to be used with HIDEELS.

Some features were removed from the training and testing sets, namely: source and destination IP addresses, source and

TABLE 9. A comparison of the average performance metrics between individual ML models and the EL model.

Model	<i>t</i>	<i>acc</i>	<i>p</i>	<i>r</i>	<i>f₁</i>
All	301.0543	99.5361%	99.5235%	99.7143%	99.6169%
EL	327.5007	99.3840%	99.5912%	99.7349%	99.6629%

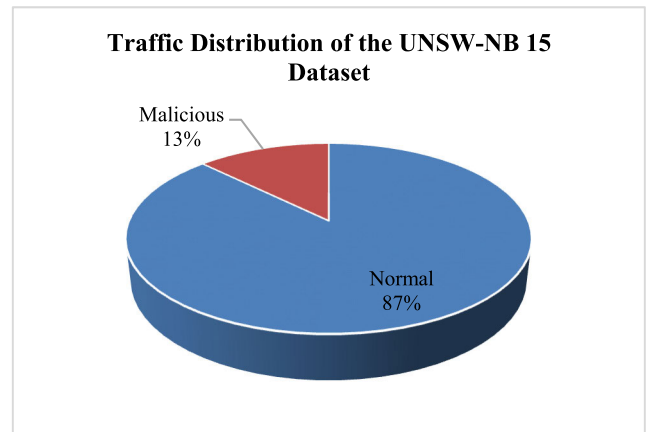


FIGURE 11. Traffic distribution of the UNSW-NB 15 dataset.

destination ports, and record start and last time. In fact, those features were removed by the creator of the training and testing dataset, and we opted to use these two subsets as they were crafted by their creator. Also, during early consideration of these two datasets, we noticed the addition of a new feature, named “rate”, that was not part of the original dataset. Thus, this “rate” feature is removed from the training and testing sets. The nominal “attack_cat” feature is removed, as it represents the name of the nominal value of the type of traffic, such as normal, Fuzzers, Analysis, Backdoors, DoS Exploits, etc.; still, there exists the label feature which classifies the type of the traffic as 0 to indicate normal or 1 to indicate malicious traffic. As a result, the full dataset as well as both the training and testing dataset comprise 42 features.

Three of the remaining features are nominal features, those are listed in Table 10. As per the type of each nominal feature, OHE is used to encode those features into numerical ones. The result is increasing the number of features by 151 and removing the original three nominal features, which total to 190 total features in the dataset.

Afterwards, normalization on the remaining features is conducted. Table 11 lists 37 features that were normalized. The values listed in Table 11 show that before normalization the ranges the features were very big, and show how normalization changed all the values of the float datatype and their values are bounded between 0 and 1.

We opted to run the SelectFromModel FS approach as it provided the least FS time while working with the KDD Cup 1999 dataset. The FS step results in selecting 71 features, out of the 190 features, in about 83 seconds.

The third dataset is the TON_IoT Network Dataset [63], [64], [65], [66], [67], [68], [69], [70], which consists of nearly

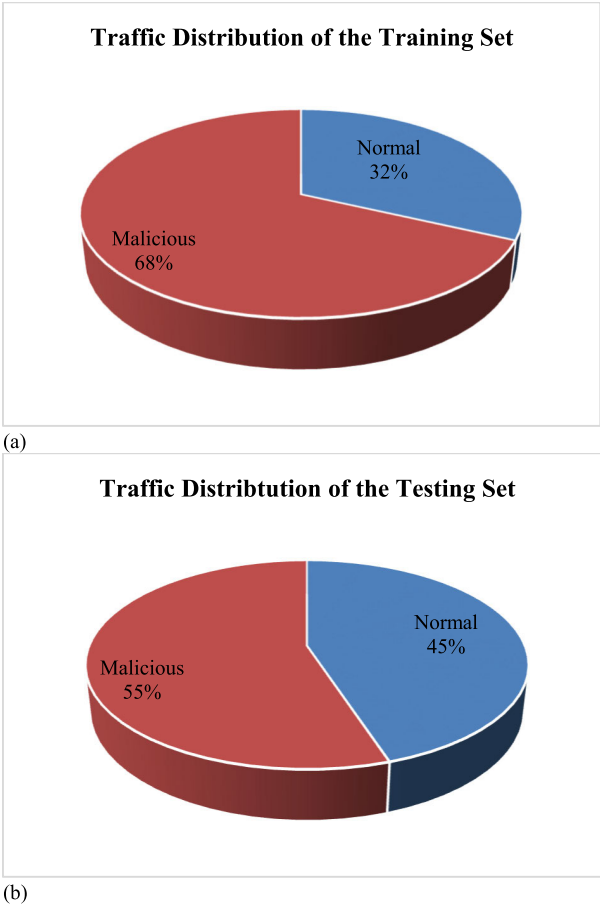


FIGURE 12. Traffic distribution of the (a) training set and (b) testing set.

TABLE 10. The nominal features of the UNSW-NB15 dataset and the emergent encoded features.

Name	Unique Values	Description	Encoding	Result
proto	131	Transaction protocol	OHE	131
state	7	Indicates to the state and its dependent protocol.		7
service	13	Name of the service		13
Total Encoded Features				151
Excluded Features				3
Total Number of Features after Encoding				190

22.3 million records which are organized in 46 features. The TON_IoT dataset is imbalanced, such that nearly 4% of the traffic is normal while the remaining 96% is malicious traffic as shown in Figure 13.

Some of the 46 features contain a huge amount of missing data, those are listed in Table 12. As shown in Table 13 the percentage of the missing values in those 23 features is greater than 75%. Thus, those features are eliminated from the dataset. As done in the previous dataset, also eliminated the first five features that represent the timestamp, source and destination IP addresses, and source and destination port

TABLE 11. Normalization results on the numeric features of the UNSW-NB15 dataset.

Feature	Before Normalization		After Normalization	
	Datatype	Range	Datatype	Range
Dur	Float	59.999989	Float	1
Spkts	Integer	10646	Float	1
Dpkts	Integer	11018	Float	1
Sbytes	Integer	14355774	Float	1
dbytes	Integer	14657531	Float	1
sttl	Integer	255	Float	1
dttl	Integer	253	Float	1
sload	Float	5268000256	Float	1
dload	Float	20821108	Float	1
sloss	Integer	5319	Float	1
dloss	Integer	5507	Float	1
sinpkt	Float	60009.992	Float	1
dinpkt	Float	57739.24	Float	1
sjit	Float	1483830.917	Float	1
djit	Float	463199.2401	Float	1
swin	Integer	255	Float	1
stcpb	Integer	4294949667	Float	1
dtcpb	Integer	4294880717	Float	1
dwin	Integer	255	Float	1
tcprtt	Float	3.821465	Float	1
synack	Float	3.226788	Float	1
ackdat	Float	2.928778	Float	1
smean	Integer	1504	Float	1
dmean	Integer	1500	Float	1
trans_depth	Integer	131	Float	1
response_body_len	Integer	5242880	Float	1
ct_srv_src	Integer	63	Float	1
ct_state_ttl	Integer	6	Float	1
ct_dst_ltm	Integer	59	Float	1
ct_src_dport_ltm	Integer	59	Float	1
ct_dst_sport_ltm	Integer	38	Float	1
ct_dst_src_ltm	Integer	63	Float	1
is_ftp_login	Integer	2	Float	1
ct_ftp_cmd	Integer	2	Float	1
ct_flw_http_mthd	Integer	16	Float	1
ct_src_ltm	Integer	60	Float	1
ct_srv_dst	Integer	62	Float	1

numbers, in addition to the last feature that represents the nominal value of the traffic type, namely “type”. In total 17 features are remained.

From the remaining features, two of them are of the nominal datatype then encoded them using OHE, as shown in Table 13. After encoding the nominal features, the resulting dataset will contain 31 features.

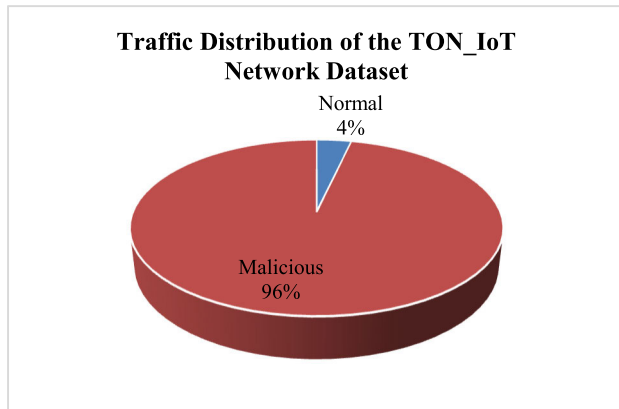


FIGURE 13. Traffic distribution of the TON_IoT Network dataset.

TABLE 12. TON_IOT network features that contain missing values more than 75%.

Feature	% Missing Values
Service	76%
dns_query	93%
dns_AA	93%
dns_RD	93%
dns_RA	93%
dns_rejected	93%
ssl_version	100%
ssl_cipher	100%
ssl_resumed	100%
ssl_established	100%
ssl_subject	100%
ssl_issuer	100%
http_trans_depth	100%
http_method	100%
http_uri	100%
http_referrer	100%
http_version	100%
http_user_agent	100%
http_orig_mime_types	100%
http_resp_mime_types	100%
weird_name	100%
weird_addl	100%
weird_notice	100%

TABLE 13. Nominal features of the TON_IOT network dataset, the encoding scheme used, and the emergent features.

Name	Unique Values	Description	Encoding	Result
proto	3	Transport layer protocols of flow connections	OHE	3
conn_state	13	Connection states,		13
Total Encoded Features				16
Excluded Features				2
Total Number of Features after Encoding				31

Table 14 summarizes the list of numeric features that need to be normalized for their values to be bounded between 0 and 1.

The SelectFromModel FS approach selected the top 12 features from the list of features to be included in model training and testing.

TABLE 14. Normalization of the numeric features of the TON_IOT network dataset.

Feature	Before Normalization		After Normalization	
	Datatype	Range	Datatype	Range
duration	Float	93516.93	Float	1
src_bytes	Integer	4.24E+09	Float	1
dst_bytes	Integer	6.17E+09	Float	1
missed_bytes	Integer	5.17E+09	Float	1
src_pkts	Integer	2570179	Float	1
src_ip_bytes	Integer	1.05E+09	Float	1
dst_pkts	Integer	3238855	Float	1
dst_ip_bytes	Integer	1.75E+08	Float	1
dns_qclass	Integer	32769	Float	1
dns_qtype	Integer	27136	Float	1
dns_rcode	Integer	5	Float	1
http_request_body_len	Integer	2338	Float	1
http_response_body_len	Integer	13424384	Float	1
http_status_code	Integer	404	Float	1

TABLE 15. Record distribution in the training and testing set of the TON_IOT network dataset.

Dataset	Normal		Malicious		Total
	Count	%	Count	%	
Training/Testing	50000	24%	161043	76%	211043
Training	35455	24%	112275	76%	147730
Testing	14545	23%	48768	77%	63313

The TON_IoT provide nearly 1% of its records for both the training and testing sets, which is equal to 211043 records, only 50000 of them are normal traffic records which weighs 24% of the subset, and the remaining 76%, which equals 161043 records, represents malicious traffic records. This subset is split into two subsets, namely the training set which contains 70% of the records, that is 147,730 records, and 30% of the records, that is 63,313, for the testing set. Inside each of the training set and testing set, the original distribution of normal and malicious traffic of the original subset is preserved, which is 24% to 76% resp. The details of record distribution in the training set and testing set is listed in Table 15.

We set the experiments to run on the remaining two datasets, and compared the results of accuracy, precision, recall, and f1-score of each dataset. The results of running each of the five ML models, on an individual basis, using the UNSW-NB 15 dataset are listed in Table 16.

The individual model performance using the TON_IoT network dataset is listed in Table 17.

Both Tables 16 and 17 show that the DT classifier has the best performance in terms of accuracy, precision, recall, and f1-score using the two datasets.

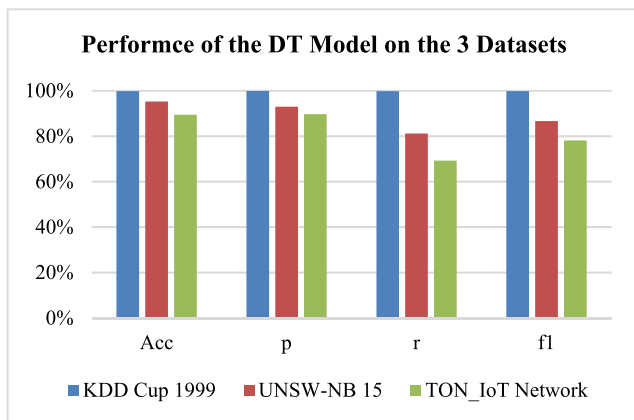
However, there are important things to notice in the results that are shown in Tables 16 and 17. First, the DT model

TABLE 16. Individual ML model performance using the UNSW-NB 15 dataset.

Model	acc	p	r	f ₁
LR	93.11%	92.62%	78.70%	85.09%
kNN	94.01%	92.90%	79.62%	85.75%
DT	95.27%	92.95%	81.15%	86.65%
NB	92.32%	90.08%	78.44%	83.86%
SVM	92.50%	92.33%	78.61%	84.92%

TABLE 17. Individual ML model using the TON_IOT network dataset.

Model	acc	p	r	f ₁
LR	86.98%	88.95%	67.05%	76.46%
kNN	87.92%	89.30%	67.35%	76.79%
DT	89.44%	89.72%	69.21%	78.14%
NB	86.72%	88.30%	65.39%	75.14%
SVM	86.88%	88.57%	66.32%	75.85%

**FIGURE 14.** Performance of the DT model on the three datasets.

always outperforms the other model in terms of accuracy, precision, recall, and f1-score, while the NB algorithm always at the bottom of the list.

Second, performance of the individual models using the KDD Cup 1999 is much better than those of the UNSW-NB 15 and the TON_IoT Network dataset as shown in Figure 14, which shows that using the DT algorithm, the results of the KDD Cup 1999 surpass those of both the UNSW-NB 15 and TON_IoT Network datasets.

The main reason behind the best results of the KDD Cup 1999 is that it is much balanced than the other two datasets. The normal to malicious distribution in the KDD Cup 1999 dataset is 60% to 40%, which is a nearly balanced dataset. On the other hand, in the UNSW-NB 15, the distribution is 87% to 13%, and it is 4% to 96% in the TON_IoT Network dataset.

It is also important to notice that the behavior of the models using the three datasets would be different. The performance

TABLE 18. Confusion matrix of the HIDEELS using the KDD cup 1999 dataset.

Actual / Predicted	Positive	Negative
Positive	TP = 72,600	FN = 100
Negative	FP = 86	TN = 72,800

TABLE 19. Ablation of HIDEELS.

Ablation	acc	p	r	f ₁
None	99.384%	99.5912%	99.7349%	99.6629%
DT	99.18%	99.40%	99.57%	99.48%
kNN	99.21%	99.43%	99.60%	99.51%
LR	99.27%	99.50%	99.64%	99.57%
SVM	99.31%	99.54%	99.68%	99.61%
NB	99.32%	99.55%	99.69%	99.62%

of the models using the KDD Cup 1999 and UNSW-NB 15 datasets tends to better detect normal traffic than malicious traffic, since the percentage of normal traffic is much higher than the malicious traffic. On the other hand, using the TON_IoT Network dataset, the model tends to detect malicious traffic more accurately, since it is the dominating class in the dataset.

Another reason for the low performance of TON_IoT is the number of features that were used in the models. Most of the TON_IoT features were eliminated because they contain a huge amount of missing values. Among the nearly 22 million records, more than 95% of them were missing, thus, they were eliminated. Despite that this act reduced the dimensionality of the dataset, however it deprived the ML models from benefitting from a great volume of information that could have made the difference if they existed.

In order to make sure that HIDEELS model is not suffering overfitting, the confusion matrix that shows the TP, TN, FP, and FN values of the KDD Cup 1999 are included in table 18, where 72,600 attacks were correctly detected, while only 100 attacks were missed, i.e., they were classified as normal rather than being classified as attacks. Also, 86 normal traffic were incorrectly flagged as attack, and 72,800 normal traffic were correctly detected.

As per the previous results, the confusion matrix and the TP, TN, FP, and FN values show that HIDEELS is capable of performing appropriate, or nearly perfect, predictions using the KDD Cup 1999. The low FP value, i.e., 86, explains that HIDEELS is capable of producing a very few numbers of false alarms, that is almost negligible. The high f1-score shows HIDEELS balanced capability of catching attacks and avoiding false alarms, which encompasses a discussion of the operational consequences of false alarms, including alert fatigue, disruption of workflow, and an increase in mean time to resolution (MTTR).

Table 19 shows the ablation study of the HIDEELS, which configured by removing a ML model at a time, rerunning the ablated HIDEELS, and record the results.

The results listed in Table 19 show that the ablation of DT results with the least accuracy, precision, recall, and f1-score.

TABLE 20. Comparison HIDEELS model with current research.

Ref.	Model	Accuracy	Current Paper
[30]	NB	71.00%	97.53%
	SVM	97.29%	98.83%
[31]	LR	87.05%	98.39%
	kNN	98.64%	99.26%
	DT	99.21%	99.54%
	NB	73.90%	97.53%
	SVM	89.31%	98.83%
[32]	DT	99.90%	99.54%
[33]	LR	92.40%	98.39%
	kNN	98.80%	99.26%
	DT	99.30%	99.54%
	NB	87.10%	97.53%
	EL	96.70%	99.38%
[35]	SVM	99.11%	98.83%
[37]	kNN	98.01%	99.26%
	DT	98.25%	99.54%
	SVM	95.45%	98.83%
	EL	98.26%	99.38%
[39]	LR	91.78%	98.39%
	kNN	98.74%	99.26%
	DT	99.96%	99.54%
	NB	86.68%	97.53%
	SVM	91.69%	98.83%
[40]	kNN	95.88%	99.26%
	EL	96.43%	99.38%
[48]	kNN	98.97%	99.26%
	DT	99.41%	99.54%
	EL	99.73%	99.38%
[49]	LR	81%	98.39%
	NB	89%	97.53%
	SVM	99%	98.83%

On the other extreme, the ablation of NB results with the least effect of in terms of accuracy, precision, recall, and f1-score.

D. COMPARISON WITH OTHER WORK

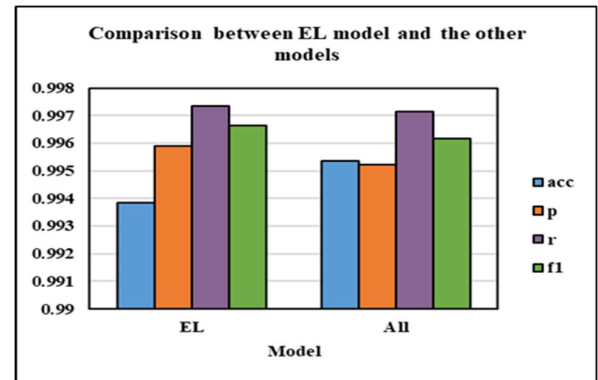
In Table 20 and figure 15, the results are compared and obtained in terms of accuracy with the results that are found in the literature.

The results presented in Table 20 demonstrate the superiority of the models employed in this study compared to their counterparts in the existing literature. The construction of the model and the inclusion of the feature selection step enhanced our models' accuracy.

Additionally, the model was modified from a multi-label selection approach to a single-label selection approach. Some may contend that this step is unnecessary and could compromise the detection process. A threat is fundamentally a threat, irrespective of its kind or type.

The current dataset exhibits an imbalance, with the frequency of malicious attacks significantly lower than that of normal attacks. The dataset's imbalance affects prediction accuracy, as accuracy is inherently influenced by dataset balance. By consolidating all types of malicious traffic into a single category, the dataset is balanced, thereby enhancing the accuracy of predictions.

A summary table 21 comparing HIDEELS with recent IDS/DevSecOps frameworks to better highlight its novelty.

**FIGURE 15.** Comparison HIDEELS model with current research.

E. EXPANSION OF HIDEELS TO KUBERNETES AND CLOUD-NATIVE ENVIRONMENTS

The objective of the proposed HIDEELS framework is to develop an advanced security solution that can be incorporated into the configuration management phase of the DevOps pipeline. The proposed HIDEELS is inherently expandable to accommodate contemporary container orchestration technologies, including Kubernetes, as well as to be adaptable for cloud-native settings. The growing implementation of microservices and containerization presents several benefits and considerations for integrating HIDEELS into these frameworks.

Microservices and container orchestration facilitate the deployment of HIDEELS components as autonomous, scalable services. This modular deployment offers flexibility and scalability for updates without disturbing the entire workflow. Regular updates of the HIDEELS components are crucial for the model's sustainability, as well as for self-tuning and auto-configuration to address emerging attack vectors and maintain vigilance against new threats.

The cloud-native characteristics of container orchestration systems, such as Kubernetes, enable HIDEELS to autonomously adjust security monitoring and response components according to workload demands. This dynamic scalability preserves the performance and resilience of HIDEELS in dynamically evolving contexts.

Cloud computing offers viable solutions for logging, identity management, and usage reporting, enabling HIDEELS to significantly benefit from the services provided by cloud service providers. This enhances the security of HIDEELS and the monitoring services it offers, as it is an integral component of the configuration management in DevSecOps in parallel sensitive consideration for cybersecurity ethical [71]. To ensure the sustainability of the enhanced monitoring system, the framework emphasizes long-term operability through automated scaling, reduced configuration overhead, and maintainable observability pipelines that remain resilient under continuous integration and deployment workflows.

TABLE 21. Comparison of HIDEELS with recent IDS/DevSecOps frameworks.

Ref	Model	Accuracy	Current Paper (Novelty: HIDEELS)
[46]	Falco (CNCF Runtime IDS)	Rule-based (no ML accuracy)	HIDEELS extends beyond rule-based runtime detection with ML-driven anomaly and adversarial robustness.
[41]	Flora et al. (Microservice IDS Benchmark)	70–90% (varies by method)	HIDEELS introduces integrated continuous monitoring with adaptive anomaly detection, not just benchmarks.
[42]	Janabi et al. (SDN IDS Survey)	Survey (no accuracy focus)	HIDEELS provides operational validation in DevOps, unlike survey-only contributions.
[43]	Adversarial NIDS Survey	No metrics (conceptual)	HIDEELS incorporates adversarial robustness into deployed IDS, bridging theory to practice.
[38]	Fawzy et al. (DADF Framework)	≈80% (prototype evaluation)	HIDEELS validates scalability in DevOps environments, surpassing prototype-level results.
[44]	Yuan et al. (Robust IDS)	90–95% with adversarial training	HIDEELS balances adversarial robustness with DevOps CI/CD integration, unlike purely ML-centric work.
[50]	ML in Containers (DevOps Study)	≈85% anomaly detection	HIDEELS unifies container telemetry with DevOps pipeline metrics, providing broader coverage.
[45]	Olaoye et al. (Adversarial ML for IDS)	Variable accuracy (experimental)	HIDEELS operationalizes adversarial defenses within continuous monitoring, not only simulations.
[47]	Vendor Reports (Sysdig, SentinelOne)	Not disclosed (proprietary)	HIDEELS is open, research-driven, with reproducible results, unlike vendor-locked solutions.
[1]	HIDEELS (Current Paper)	99.3840%	Novel integration of hybrid IDS with DevOps continuous monitoring phase, combining anomaly detection, adversarial resilience, and scalability.

F. PRACTICAL IMPLEMENTATION OF HIDEELS

1. More labor for the process HIDEELS works well with cloud-native systems, with only a little reduction in performance.

Telemetry agents that don't take up much space and machine learning inference pipelines that work quickly cut down on the time it takes to run a program. Using streaming log and metric analysis instead of batch processing to find abnormalities lowers the memory and CPU expenses of containerized microservices by 5–8%.

The hybrid method only switches on more complex ML models when it sees something suspicious, which saves processing power.

2. HIDEELS works well with cloud-native workflows, which is perfect for DevOps teams that need to deal with workloads that change. With Kubernetes or another container orchestration technology, the architecture makes it easier to scale detection modules horizontally. You may extend the pipeline in real time based on activity if you add telemetry input, log parsing, and machine learning-based anomaly detection modules to message queues or event streams like Kafka or RabbitMQ. This protects the monitoring system from things that happen all the time in a continuous integration and delivery pipeline.

3. HIDEELS works great with DevOps CI/CD technologies such as GitHub Actions, Jenkins, GitLab CI, and Azure DevOps. The points of connection:

- Pre-deployment assessments: before deployment, evaluations while developing and testing, look at the logs and data in the pipeline to detect problems early.
- Runtime monitoring hooks: hooks for keeping an eye on the program while it's running to keep a watch on services that are running, use lightweight agents in Kubernetes pods or virtual machines.
- Automated alerts and mitigation: you may speed up the process of correcting problems by using HIDEELS warnings with workflow automation tools like GitLab issue creation, Slack notifications, and Kubernetes auto-scaling triggers.

4. Pragmatic Considerations to enhance deployment efficacy, companies should:

- To make anomaly detection and deployment work better, companies should set baseline behavioral indicators for microservices.
- Improve machine learning models so they can strike the correct balance between sensitivity and false-positive rates, especially for short-term jobs.
- Continuously update detection criteria and retrain. Change the rules for finding changes in the CI/CD process and train the models again to keep them working.

5. Expected Benefits

- DevOps pipelines show up immediately away and don't slow down performance.
- Scalable identification through various clouds locations, services, and clusters.
- Flawless incorporation with current CI/CD workflows: Works nicely with current CI/CD workflows to automatically deal with problems and risks.

Figure 16 shows the suggested structure for Practical Deployment of HIDEELS with current CI/CD workflows.

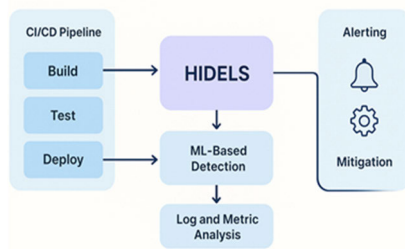


FIGURE 16. Practical deployment of HIDEELS with current CI/CD workflows.

V. CONCLUSION AND FUTURE WORK

This study introduces an innovative hybrid model that amalgamates machine learning and evolutionary learning techniques to provide an intrusion detection system for incorporation into the continuous monitoring phase of DevOps. The innovative hybrid methodology is referred to as HIDEELS.

Five machine learning algorithms: Logistic Regression (LR), k-Nearest Neighbors (kNN), Decision Tree (DT), Naive Bayes (NB), and Support Vector Machine (SVM) are used. We utilized the KDD Cup 1999 dataset, which is extensively employed in the domain of intrusion detection and machine learning models. DT outperformed all other models individually, achieving an average accuracy of 99.5361%, an average precision of 99.5235%, a recall of 99.7143%, and an average F1-score of 99.6169333%.

An EL model was subsequently developed employing k-fold cross-validation. The model was developed using the same machine learning models that were evaluated separately. The findings indicate that the EL model attained an average accuracy of 99.4%, an average precision of 99.6%, an average recall of 99.7%, and an average F1-score of 99.7%. Furthermore, the EL model surpassed the ML models when executed independently, demonstrating the EL model's superiority in achieving superior outcomes. The EL model can consolidate superior-performing models while simultaneously mitigating the impact of underperforming models.

This paper aims to illustrate how various machine learning models can collaboratively function within an ensemble learning configuration to augment performance and enhance system predictability, even when individual models yield sub-optimal results for similar problems. This EL technique offers a highly interpreted and computationally efficient solution suitable for practical, real-world applications, particularly in resource-constrained environments. The utilization of sophisticated models, including large language models (LLMs), necessitates a resource-intensive environment that exceeds the parameters of this article. The approach presented in this paper offers computational efficiency in resource-constrained systems. Furthermore, it upholds privacy and security by not necessitating the transmission of potentially sensitive log data

to external, third-party APIs. This is a significant issue in numerous businesses and regulated settings.

Future research may involve the evaluation of additional machine learning models, including artificial neural networks (ANN). Additionally, the security system of DevSecOps can incorporate capabilities like malware detection, phishing detection, and spam detection. Moreover, future research may prioritize the integration of deep learning (DL) methodologies in the intrusion detection system (IDS). Additionally, alternative methodologies may be evaluated, including the greedy algorithm, grey wolf optimizer (GWO), and defeasible logic.

Furthermore, the application of LLMs can be examined regarding their efficacy in anomaly prediction and their capacity to offer solutions and recommendations for enhancing enterprise security, utilizing a fully integrated knowledge base supported by insights derived from generative pre-trained transformer (GPT) models.

Subsequent study ought to enhance the feature selection techniques utilized herein by integrating systematic hyperparameter optimization approaches, including grid search, random search, or Bayesian optimization. These methodologies would provide a more thorough examination of parameter combinations, potentially enhancing model correctness, robustness, and generalizability. Expanding the framework in this manner would enhance the feature selection outcomes reported in the present study and exhibit additional methodological sophistication.

ACKNOWLEDGMENT

The authors would like to thank Zarqa University for their knowledge and insight, which were very helpful to the research.

COMPETING INTERESTS

There are no competing interests among the authors.

AUTHOR CONTRIBUTIONS

All the authors have contributed in an equal manner.

DATA AVAILABILITY

Data is available based on personal request.

REFERENCES

- [1] C. Ebert, G. Gallardo, J. Hernantes, and N. Serrano, "DevOps 2.0," *IEEE Softw.*, vol. 42, no. 2, pp. 24–32, Mar. 2025, doi: 10.1109/MS.2025.3525768.
- [2] M. S. Khan, A. W. Khan, F. Khan, M. A. Khan, and T. K. Whangbo, "Critical challenges to adopt DevOps culture in software organizations: A systematic review," *IEEE Access*, vol. 10, pp. 14339–14349, 2022, doi: 10.1109/ACCESS.2022.3145970.
- [3] R. Subramanya, S. Sierla, and V. Vyatkin, "From DevOps to MLOps: Overview and application to electricity market forecasting," *Appl. Sci.*, vol. 12, no. 19, p. 9851, Sep. 2022, doi: 10.3390/app12199851.
- [4] M. A. López-Peña, J. Díaz, J. E. Pérez, and H. Humanes, "DevOps for IoT systems: Fast and continuous monitoring feedback of system availability," *IEEE Internet Things J.*, vol. 7, no. 10, pp. 10695–10707, Oct. 2020, doi: 10.1109/JIOT.2020.3012763.

- [5] G. Cascavilla, "The rise of cybercrime and cyber-threat intelligence: Perspectives and challenges from law enforcement," *IEEE Secur. Privacy*, vol. 23, no. 1, pp. 17–26, Jan. 2025, doi: [10.1109/MSEC.2024.3407859](https://doi.org/10.1109/MSEC.2024.3407859).
- [6] D. Fujs, S. Vrhovec, and D. Vavpotič, "Towards personalized user training for secure use of information systems," *Int. Arab J. Inf. Technol.*, vol. 19, no. 3, pp. 307–313, 2022, doi: [10.34028/iajit/19/3/3](https://doi.org/10.34028/iajit/19/3/3).
- [7] J. Ababneh, M. Mohammed Alnemari, H. Attar, H. A. Alghamdi, M. Al-Shawaheen, B. Saqarat, L. Abu Romman, and M. Iqtait, "Cybersecurity ethical aspects (CEA)," *IEEE Access*, vol. 13, pp. 212443–212468, 2025, doi: [10.1109/ACCESS.2025.3641180](https://doi.org/10.1109/ACCESS.2025.3641180).
- [8] O. Surakhi, A. García, M. Jamoos, and M. Alkhanafseh, "The intrusion detection system by deep learning methods: Issues and challenges," *Int. Arab J. Inf. Technol.*, vol. 19, no. 3A, pp. 501–513, 2022, doi: [10.34028/iajit/19/3A/10](https://doi.org/10.34028/iajit/19/3A/10).
- [9] H. Ghaffari, "Choosing decision tree-based boundary patterns in the intrusion detection systems with large data sets," *Int. Arab J. Inf. Technol.*, vol. 19, no. 3, pp. 459–468, 2022, doi: [10.34028/iajit/19/3/10](https://doi.org/10.34028/iajit/19/3/10).
- [10] R. Chowdhury, S. Sen, A. Roy, and B. Saha, "An optimal feature based network intrusion detection system using bagging ensemble method for real-time traffic analysis," *Multimedia Tools Appl.*, vol. 81, no. 28, pp. 41225–41247, May 2022, doi: [10.1007/s11042-022-12330-3](https://doi.org/10.1007/s11042-022-12330-3).
- [11] H. Dong, G. Frusque, Y. Zhao, E. Chatzi, and O. Fink, "NNG-mix: Improving semi-supervised anomaly detection with pseudo-anomaly generation," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 6, pp. 10635–10647, Jun. 2025, doi: [10.1109/TNNLS.2024.3497801](https://doi.org/10.1109/TNNLS.2024.3497801).
- [12] Y. Kang, B. Yu, and Z. Xu, "Interpretable multiattribute predictive analysis model based on rough fuzzy sets and logistic regression," *IEEE Trans. Fuzzy Syst.*, vol. 31, no. 10, pp. 3522–3532, Oct. 2023, doi: [10.1109/TFUZZ.2023.3260843](https://doi.org/10.1109/TFUZZ.2023.3260843).
- [13] V. K. Daliya and T. K. Ramesh, "A cloud-based optimized ensemble model for risk prediction of diabetic progression—An Azure machine learning perspective," *IEEE Access*, vol. 13, pp. 11560–11575, 2025, doi: [10.1109/ACCESS.2025.3528033](https://doi.org/10.1109/ACCESS.2025.3528033).
- [14] X. Cheng, M. Liao, and J. Liu, "Geographical origin identification of panax notoginseng using a modified K-nearest neighbors model with near-infrared spectroscopy," *IEEE Access*, vol. 13, pp. 13832–13846, 2025, doi: [10.1109/ACCESS.2025.3530766](https://doi.org/10.1109/ACCESS.2025.3530766).
- [15] J. Bai, Y. Lian, Y. Wang, J. Ren, Z. Xiao, H. Zhou, and L. Jiao, "An interpretable explanation approach for signal modulation classification," *IEEE Trans. Instrum. Meas.*, vol. 73, pp. 1–13, 2024, doi: [10.1109/TIM.2024.3381706](https://doi.org/10.1109/TIM.2024.3381706).
- [16] R. Rivera-López and H. G. Ceballos, "A differential-evolution-based approach to extract univariate decision trees from black-box models using tabular data," *IEEE Access*, vol. 12, pp. 169850–169868, 2024, doi: [10.1109/ACCESS.2024.3498907](https://doi.org/10.1109/ACCESS.2024.3498907).
- [17] L. Guo, R. Hao, J. Yu, and M. Yang, "Privacy-preserving Naïve Bayesian classification for health monitoring systems," *IEEE Trans. Ind. Informat.*, vol. 20, no. 10, pp. 11622–11634, Oct. 2024, doi: [10.1109/TII.2024.3409452](https://doi.org/10.1109/TII.2024.3409452).
- [18] S. Naiem, A. E. Khedr, A. M. Idrees, and M. I. Marie, "Enhancing the efficiency of Gaussian Naïve Bayes machine learning classifier in the detection of DDOS in cloud computing," *IEEE Access*, vol. 11, pp. 124597–124608, 2023, doi: [10.1109/ACCESS.2023.3328951](https://doi.org/10.1109/ACCESS.2023.3328951).
- [19] Y. Chen, X. Cui, Y. Liu, and X. Cui, "An evaluation method of the anti-modeling-attack capability of PUFs," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 1773–1788, 2023, doi: [10.1109/TIFS.2023.3254434](https://doi.org/10.1109/TIFS.2023.3254434).
- [20] M. Si, Y. Wang, H. Siljak, C. Seow, and H. Yang, "A lightweight CIR-based CNN with MLP for NLOS/LOS identification in a UWB positioning system," *IEEE Commun. Lett.*, vol. 27, no. 5, pp. 1332–1336, May 2023, doi: [10.1109/LCOMM.2023.3260953](https://doi.org/10.1109/LCOMM.2023.3260953).
- [21] S. A. Licorish, "Understanding the effect of agile practice quality on software product quality," *IEEE Trans. Softw. Eng.*, vol. 51, no. 2, pp. 650–662, Feb. 2025, doi: [10.1109/TSE.2025.3532502](https://doi.org/10.1109/TSE.2025.3532502).
- [22] C. Ebert, G. Gallardo, J. Hernantes, and N. Serrano, "DevOps," *IEEE Softw.*, vol. 33, no. 3, pp. 94–100, May 2016, doi: [10.1109/MS.2016.68](https://doi.org/10.1109/MS.2016.68).
- [23] K. Boisrond, P. M. Tardif, and F. Jaafar, "Ensuring the integrity, confidentiality, and availability of IoT data in Industry 5.0: A systematic mapping study," *IEEE Access*, vol. 12, pp. 107017–107045, 2024, doi: [10.1109/ACCESS.2024.3434618](https://doi.org/10.1109/ACCESS.2024.3434618).
- [24] S. Rafi, W. Yu, M. A. Akbar, A. Alsanad, and A. Gumaei, "Prioritization based taxonomy of DevOps security challenges using PROMETHEE," *IEEE Access*, vol. 8, pp. 105426–105446, 2020, doi: [10.1109/ACCESS.2020.2998819](https://doi.org/10.1109/ACCESS.2020.2998819).
- [25] R. N. Rajapakse, M. Zahedi, M. A. Babar, and H. Shen, "Challenges and solutions when adopting DevSecOps: A systematic review," *Inf. Softw. Technol.*, vol. 141, Jan. 2022, Art. no. 106700, doi: [10.1016/j.infsof.2021.106700](https://doi.org/10.1016/j.infsof.2021.106700).
- [26] V. Mohan and L. B. Othmane, "SecDevOps: Is it a marketing buzzword?—Mapping research on security in DevOps," in *Proc. 11th Int. Conf. Availability, Rel. Secur. (ARES)*, Salzburg, Austria, Aug. 2016, pp. 542–547, doi: [10.1109/ARES.2016.92](https://doi.org/10.1109/ARES.2016.92).
- [27] L. Prates, J. Faustino, M. Silva, and R. Pereira, "DevSecOps metrics," in *Information Systems: Research, Development, Applications, Education (SIGSAND/PLAIS 2019)* (Lecture Notes in Bus. Information Processing), S. Wrycza and J. Małankowski, Eds., Cham, Switzerland: Springer, 2019, pp. 77–90, doi: [10.1007/978-3-030-29608-7_7](https://doi.org/10.1007/978-3-030-29608-7_7).
- [28] M. A. Aljohani and S. S. Alqahtani, "A unified framework for automating software security analysis in DevSecOps," in *Proc. Int. Conf. Smart Comput. Appl. (ICSCA)*, Feb. 2023, pp. 1–6, doi: [10.1109/ICSCA57840.2023.10087568](https://doi.org/10.1109/ICSCA57840.2023.10087568).
- [29] R. Mao, H. Zhang, Q. Dai, H. Huang, G. Rong, H. Shen, L. Chen, and K. Lu, "Preliminary findings about DevSecOps from grey literature," in *Proc. IEEE 20th Int. Conf. Softw. Quality, Rel. Secur. (QRS)*, Dec. 2020, pp. 450–457, doi: [10.1109/QRS51102.2020.00064](https://doi.org/10.1109/QRS51102.2020.00064).
- [30] A. Halimaa A. and K. Sundarakantham, "Machine learning based intrusion detection system," in *Proc. 3rd Int. Conf. Trends Electron. Informat. (ICOEI)*, Tirunelveli, India, Apr. 2019, pp. 916–920, doi: [10.1109/ICOEI.2019.8862784](https://doi.org/10.1109/ICOEI.2019.8862784).
- [31] I. Abrar, Z. Ayub, F. Masoodi, and A. M. Bamhdi, "A machine learning approach for intrusion detection system on NSL-KDD dataset," in *Proc. Int. Conf. Smart Electron. Commun. (ICOSEC)*, Trichy, India, Sep. 2020, pp. 919–924, doi: [10.1109/ICOSEC49089.2020.9215232](https://doi.org/10.1109/ICOSEC49089.2020.9215232).
- [32] R. A. Disha and S. Waheed, "Performance analysis of machine learning models for intrusion detection system using Gini impurity-based weighted random forest (GIWRF) feature selection technique," *Cybersecurity*, vol. 5, no. 1, Jan. 2022, Art. no. 41, doi: [10.1186/s42400-021-00103-8](https://doi.org/10.1186/s42400-021-00103-8).
- [33] M. B. Pranto, M. H. A. Ratul, M. M. Rahman, I. J. Diya, and Z.-B. Zahir, "Performance of machine learning techniques in anomaly detection with basic feature selection strategy—A network intrusion detection system," *J. Adv. Inf. Technol.*, vol. 13, no. 1, pp. 36–44, 2022, doi: [10.12720/jait.13.1.36-44](https://doi.org/10.12720/jait.13.1.36-44).
- [34] S. Kumar, S. Gupta, and S. Arora, "A comparative simulation of normalization methods for machine learning-based intrusion detection systems using KDD Cup'99 dataset," *J. Intell. Fuzzy Syst.*, vol. 42, no. 3, pp. 1749–1766, 2022, doi: [10.3233/jifs-21119](https://doi.org/10.3233/jifs-21119).
- [35] M. A. Almaiah, O. Almomani, A. Alsaaidah, S. Al-Otaibi, N. Bani-Hani, A. K. A. Hwaitat, A. Al-Zahrani, A. Lutfi, A. B. Awad, and T. H. H. Aldhyani, "Performance investigation of principal component analysis for intrusion detection system using different support vector machine kernels," *Electronics*, vol. 11, no. 21, p. 3571, Nov. 2022, doi: [10.3390/electronics11213571](https://doi.org/10.3390/electronics11213571).
- [36] R. Desai and V. T. Gopalakrishnan, "Network intrusion detection through machine learning with efficient feature selection," in *Proc. 15th Int. Conf. Commun. Syst. Netw. (COMSNETS)*, Bangalore, India, Jan. 2023, pp. 797–801, doi: [10.1109/COMSNETS56262.2023.10041315](https://doi.org/10.1109/COMSNETS56262.2023.10041315).
- [37] M. A. Elsadig, "Detection of denial-of-service attack in wireless sensor networks: A lightweight machine learning approach," *IEEE Access*, vol. 11, pp. 83537–83552, 2023, doi: [10.1109/ACCESS.2023.3303113](https://doi.org/10.1109/ACCESS.2023.3303113).
- [38] A. H. Fawzy, K. Wassif, and H. Moussa, "Framework for automatic detection of anomalies in DevOps (DADF)," *J. King Saud Univ. Comput. Inf. Sci.*, vol. 35, no. 3, pp. 8–19, Mar. 2023, doi: [10.1016/j.jksuci.2023.02.010](https://doi.org/10.1016/j.jksuci.2023.02.010).
- [39] F. A. Khan, A. A. Shah, N. Alshammry, S. Saif, W. Khan, M. O. Malik, and Z. Ullah, "Balanced multi-class network intrusion detection using machine learning," *IEEE Access*, vol. 12, pp. 178222–178236, 2024, doi: [10.1109/ACCESS.2024.3503497](https://doi.org/10.1109/ACCESS.2024.3503497).
- [40] A. Abdulbory and J. S. Shin, "An incremental majority voting approach for intrusion detection system based on machine learning," *IEEE Access*, vol. 12, pp. 18972–18986, 2024, doi: [10.1109/ACCESS.2024.3361041](https://doi.org/10.1109/ACCESS.2024.3361041).
- [41] J. Flora and N. Antunes, "Evaluating intrusion detection for microservice applications: Benchmark, dataset, and case studies," *J. Syst. Softw.*, vol. 216, Oct. 2024, Art. no. 112142, doi: [10.1016/j.jss.2024.112142](https://doi.org/10.1016/j.jss.2024.112142).
- [42] A. H. Janabi, T. Kanakis, and M. Johnson, "Survey: Intrusion detection system in software-defined networking," *IEEE Access*, vol. 12, pp. 164097–164120, 2024, doi: [10.1109/ACCESS.2024.3493384](https://doi.org/10.1109/ACCESS.2024.3493384).
- [43] "Adversarial challenges in network intrusion detection," 2024, *arXiv: 2403.12345*.

- [44] X. Yuan, S. Han, W. Huang, H. Ye, X. Kong, and F. Zhang, "A simple framework to enhance the adversarial robustness of deep learning-based intrusion detection system," *Comput. Secur.*, vol. 137, Feb. 2024, Art. no. 103644, doi: [10.1016/j.cose.2023.103644](https://doi.org/10.1016/j.cose.2023.103644).
- [45] O. Olaoye, "Adversarial machine learning for robust intrusion detection systems," 2024, *arXiv: 2405.54321*.
- [46] CNCF Project Documentation. (2025). *Runtime Security Tool*. [Online]. Available: <https://falco.org>
- [47] Sysdig, SentinelOne. (2025). *Container Runtime Security Tools Reports*. [Online]. Available: <https://www.sentinelone.com>
- [48] S. A. Birahim, A. Paul, F. Rahman, Y. Islam, T. Roy, M. A. Hasan, F. Haque, and M. E. H. Chowdhury, "Intrusion detection for wireless sensor network using particle swarm optimization based explainable ensemble machine learning approach," *IEEE Access*, vol. 13, pp. 13711–13730, 2025, doi: [10.1109/ACCESS.2025.3528341](https://doi.org/10.1109/ACCESS.2025.3528341).
- [49] S. Ness, V. Eswarakrishnan, H. Sridharan, V. Shinde, N. Venkata Prasad Janapareddy, and V. Dhanawat, "Anomaly detection in network traffic using advanced machine learning techniques," *IEEE Access*, vol. 13, pp. 16133–16149, 2025, doi: [10.1109/ACCESS.2025.3526988](https://doi.org/10.1109/ACCESS.2025.3526988).
- [50] (2025). *Machine Learning-Based Anomaly Detection in Containerized Microservices Within DevOps Pipelines*. [Online]. Available: <https://www.researchgate.net>
- [51] R. Alazaidah, A. Al-qerem, M. H. Qasem, A. Al-shaikh, N. Almilli, and M. Injadat, "Feature selection in associative classification—A review and comparative analysis," in *Proc. 24th Int. Arab Conf. Inf. Technol. (ACIT)*, Dec. 2023, pp. 1–5, doi: [10.1109/ACIT58888.2023.10453798](https://doi.org/10.1109/ACIT58888.2023.10453798).
- [52] E. Casas, L. Ramos, E. Bendek, and F. Rivas, "Assessing the effectiveness of YOLO architectures for smoke and wildfire detection," *IEEE Access*, vol. 11, pp. 96554–96583, 2023, doi: [10.1109/ACCESS.2023.3312217](https://doi.org/10.1109/ACCESS.2023.3312217).
- [53] D. Dua and C. Graff. (2017). *UCI Machine Learning Repository*. Accessed: Feb. 8, 2025. [Online]. Available: <http://archive.ics.uci.edu/ml>
- [54] Q. Zhou, Z. Huang, M. Ding, and X. Zhang, "Medical image classification using light-weight CNN with spiking cortical model based attention module," *IEEE J. Biomed. Health Inform.*, vol. 27, no. 4, pp. 1991–2002, Apr. 2023, doi: [10.1109/JBHI.2023.3241439](https://doi.org/10.1109/JBHI.2023.3241439).
- [55] P. Kumar Sahu and T. Fatma, "Optimized breast cancer classification using PCA-LASSO feature selection and ensemble learning strategies with optuna optimization," *IEEE Access*, vol. 13, pp. 35645–35661, 2025, doi: [10.1109/ACCESS.2025.3539746](https://doi.org/10.1109/ACCESS.2025.3539746).
- [56] C. Mazzocca, N. Romandini, M. Mendula, R. Montanari, and P. Bellavista, "TruFLaaS: Trustworthy federated learning as a service," *IEEE Internet Things J.*, vol. 10, no. 24, pp. 21266–21281, Dec. 2023, doi: [10.1109/JIOT.2023.3282899](https://doi.org/10.1109/JIOT.2023.3282899).
- [57] J. Li, J. Ma, O. M. Omisore, Y. Liu, H. Tang, P. Ao, Y. Yan, L. Wang, and Z. Nie, "Noninvasive blood glucose monitoring using spatiotemporal ECG and PPG feature fusion and weight-based choquet integral multi-model approach," *IEEE Trans. Neural Netw. Syst.*, vol. 35, no. 10, pp. 14491–14505, Oct. 2024, doi: [10.1109/TNNLS.2023.3279383](https://doi.org/10.1109/TNNLS.2023.3279383).
- [58] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in *Proc. Mil. Commun. Inf. Syst. Conf. (MilCIS)*, Nov. 2015, pp. 1–6, doi: [10.1109/MILCIS.2015.7348942](https://doi.org/10.1109/MILCIS.2015.7348942).
- [59] N. Moustafa and J. Slay, "The evaluation of network anomaly detection systems: Statistical analysis of the UNSW-NB15 data set and the comparison with the KDD99 data set," *Inf. Secur. J., Global Perspective*, vol. 25, nos. 1–3, pp. 18–31, Jan. 2016, doi: [10.1080/19393555.2015.1125974](https://doi.org/10.1080/19393555.2015.1125974).
- [60] N. Moustafa, J. Slay, and G. Creech, "Novel geometric area analysis technique for anomaly detection using trapezoidal area estimation on large-scale networks," *IEEE Trans. Big Data*, vol. 5, no. 4, pp. 481–494, Dec. 2019, doi: [10.1109/TBDDATA.2017.2715166](https://doi.org/10.1109/TBDDATA.2017.2715166).
- [61] N. Moustafa, G. Creech, and J. Slay, "Big data analytics for intrusion detection system: Statistical decision-making using finite Dirichlet mixture models," in *Data Analytics and Decision Support for Cybersecurity*, I. P. Carrascosa, H. K. Kalutara, and Y. Huang, Eds., Cham, Switzerland: Springer, 2017, pp. 127–156, doi: [10.1007/978-3-319-59439-2_5](https://doi.org/10.1007/978-3-319-59439-2_5).
- [62] M. Sarhan, S. Layeghy, N. Moustafa, and M. Portmann, "NetFlow datasets for machine learning-based network intrusion detection systems," in *Big Data Technologies and Applications (BDTA 2020, WiCON 2020)* (Lecture Notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering), Z. Deze, H. Huang, R. Hou, S. Rho, and N. Chilamkurti, Eds., Cham, Switzerland: Springer, 2021, pp. 117–135, doi: [10.1007/978-3-030-72802-1_9](https://doi.org/10.1007/978-3-030-72802-1_9).
- [63] N. Moustafa, "A new distributed architecture for evaluating AI-based security systems at the edge: Network TON_IoT datasets," *Sustain. Cities Soc.*, vol. 72, Sep. 2021, Art. no. 102994, doi: [10.1016/j.scs.2021.102994](https://doi.org/10.1016/j.scs.2021.102994).
- [64] T. M. Booi, I. Chiscop, E. Meeuwissen, N. Moustafa, and F. T. H. den Hartog, "ToN IoT-The role of heterogeneity and the need for standardization of features and attack types in IoT network intrusion datasets," *IEEE Internet Things J.*, vol. 8, no. 13, pp. 10803–10815, Jun. 2021, doi: [10.1109/JIOT.2021.3088230](https://doi.org/10.1109/JIOT.2021.3088230).
- [65] A. Alsaedi, N. Moustafa, Z. Tari, A. Mahmood, and A. Anwar, "TON_IoT telemetry dataset: A new generation dataset of IoT and IIoT for data-driven intrusion detection systems," *IEEE Access*, vol. 8, pp. 165130–165150, 2020, doi: [10.1109/ACCESS.2020.3022862](https://doi.org/10.1109/ACCESS.2020.3022862).
- [66] N. Moustafa, M. Keshky, E. Debiez, and H. Janicke, "Federated TON_IoT windows datasets for evaluating AI-based security applications," in *Proc. IEEE 19th Int. Conf. Trust, Secur. Privacy Comput. Commun. (TrustCom)*, Dec. 2020, pp. 848–855, doi: [10.1109/TRUSTCOM50675.2020.00114](https://doi.org/10.1109/TRUSTCOM50675.2020.00114).
- [67] N. Moustafa, M. Ahmed, and S. Ahmed, "Data analytics-enabled intrusion detection: Evaluations of ToN_IoT Linux datasets," in *Proc. IEEE 19th Int. Conf. Trust, Secur. Privacy Comput. Commun. (TrustCom)*, Dec. 2020, pp. 727–735, doi: [10.1109/TRUSTCOM50675.2020.00100](https://doi.org/10.1109/TRUSTCOM50675.2020.00100).
- [68] N. Moustafa, "New generations of Internet of Things datasets for cybersecurity applications based machine learning: TON_IoT datasets," in *Proc. eResearch Australasia Conf.*, 2019.
- [69] N. Moustafa, "A systemic IoT-fog-cloud architecture for big-data analytics and cyber security systems: A review of fog computing," 2019, *arXiv:1906.01055*.
- [70] J. Ashraf, M. Keshk, N. Moustafa, M. Abdel-Basset, H. Khurshid, A. D. Bakhshi, and R. R. Mostafa, "IoTBoT-IDS: A novel statistical learning-enabled botnet detection framework for protecting networks of smart cities," *Sustain. Cities Soc.*, vol. 72, Sep. 2021, Art. no. 103041, doi: [10.1016/j.scs.2021.103041](https://doi.org/10.1016/j.scs.2021.103041).
- [71] J. Ababneh et al., "Cybersecurity laws, digital crimes, and ethical considerations: A multidimensional perspective on cyber risk regulation (MPoCRR)," *J. Cloud Comp.*, vol. 14, p. 87, 2025, doi: [10.1186/s13677-025-](https://doi.org/10.1186/s13677-025-)



JAFAR ABABNEH (Member, IEEE) received the B.Sc. degree in telecommunication engineering, in 1991, the M.Sc. degree, in 2005, and the Ph.D. degree, in 2009. In 2009, he joined as the Head of computer information and network systems in information technology (IT) with WISE University, for four years. He was the Dean of the IT Faculty, WISE University, for more than five years. In March 2022, he joined the Abdul Aziz Al Ghurair School of Advanced Computing (ASAC), Luminus Technical University College (LTUC), for two years. He has been with the Cyber Security Department, Faculty of Information Technology, Zarqa University, Jordan, since January 2024. He is currently an Associate Professor. He has published many research papers, book chapters, and books in international refereed journals and conferences. His research interests include congestion and network performance, wireless and mobile network, cybersecurity, wireless sensor networks (WSNs), artificial intelligence, data mining and retrieving information, cloud computing, and E-learning.



ESAM Y. A. AL-NSOUR received the Ph.D. degree in computer science from the University of Jordan, in 2018. He is currently an Assistant Professor with the Department of Computer Science, Faculty of Information Technology, Zarqa University, Jordan. His research interests include algorithms, spatial databases, cloud computing, natural language processing, artificial intelligence, and wireless sensor networks.



ALAA AL-SHAIKH received the Ph.D. degree in computer science from the University of Jordan, in 2020. He is currently an Assistant Professor of computer science with the Department of Cybersecurity, Faculty of Information Technology, Zarqa University, Jordan. His research interests include computational intelligence, machine learning, artificial intelligence, parallel computing, distributed systems, cloud computing, algorithms, e-learning, and cybersecurity.



MOHAMMAD RASMI AL-MOUSA received the Ph.D. degree in network security from Universiti Sains Malaysia, in 2013. He is currently an Associate Professor with the Department of Cybersecurity, Faculty of IT, Zarqa University. His research interests include digital forensics, cybersecurity, E-government strategy, cloud computing, and software engineering.



AHMAD AL-ZABIN received the B.Sc. and master's degrees in computer science from Zarqa University. He is currently an Associate Developer with the PWC's Digital Services Team, Amman, Jordan. He is also a Full-Stack Developer whose areas of interest are developing dynamic websites. He has completed a variety of successful projects in the field of his expertise, while he always learns new methodologies to develop excellent quality projects.



MAHMOUD ASASSEFEH received the B.Sc. degree in telecommunication engineering from Mu'tah University, Karak, Jordan, in 1990, the master's degree in computer engineering from Yarmouk University, Irbid, Jordan, in 2004, and the Ph.D. degree in computer science from Jordan University, Amman, Jordan, in 2021. He is currently an Assistant Professor of cybersecurity with Zarqa University. He has a number of publications in a number of international journals. His research interests include network security, data mining, and the Internet of Things.



HASAN ABUALESE received the B.Sc. degree in computer science from the University of Jordan, the M.Sc. degree in software engineering from UPM, and the Ph.D. degree in software engineering from USM. He is currently an Assistant Professor of software engineering with the Prince Abdullah bin Ghazi Faculty of Information and Communication Technology, Al-Balqa Applied University, Jordan. His research interests include software testing and quality assurance, web security, and software comprehension.

...